

Assessing Process-Centered Software Engineering Environments

VINCENZO AMBRIOLA

Università di Pisa

REIDAR CONRADI

NTNU-Trondheim

and

ALFONSO FUGGETTA

Politecnico di Milano and CEFRIEL

Process-centered software engineering environments (PSEEs) are the most recent generation of environments supporting software development activities. They exploit an explicit representation of the process (called the *process model*) that specifies how to carry out software development activities, the roles and tasks of software developers, and how to use and control software development tools. A process model is therefore a vehicle to better understand and communicate the process. If it is expressed in a formal notation, it can be used to support a variety of activities such as process analysis, process simulation, and process enactment. PSEEs provide automatic support for these activities. They exploit languages based on different paradigms, such as Petri nets and rule-based systems. They include facilities to edit and analyze process models. By enacting the process model, a PSEE provides a variety of services, such as assistance for software developers, automation of routine tasks, invocation and control of software development tools, and enforcement of mandatory rules and practices. Several PSEEs have been developed, both as research projects and as commercial products. The initial deployment and exploitation of this technology have made it possible to produce a significant amount of experiences, comments, evaluations, and feedback. We still lack, however, consistent and comprehensive assessment methods that can be used to collect and organize this information. This article aims at contributing to the definition of such methods, by providing a systematic comparison grid and by accomplishing an initial evaluation of the state of the art in the field. This evaluation takes into account the systems that have been developed by the authors in the past five years, as well as the main characteristics of other well-known environments.

Categories and Subject Descriptors: D.2.6 [Software Engineering]: Programming Environments; K.6.3 [Management of Computing and Information Systems]: Software Management—*software development; software maintenance*

Authors' addresses: V. Ambriola, Dipartimento di Informatica, Università di Pisa, Corso Italia, 40, 56125 Pisa, Italy; email: ambriola@di.unipi.it; R. Conradi, Department of Computer Systems, Norwegian University of Science and Technology (NTNU), N-7034 Trondheim, Norway; email: conradi@idt.ntnu.no; A. Fuggetta, Dipartimento di Elettronica e Informazione, Politecnico di Milano, P.za Leonardo da Vinci, 32, 20133 Milano, Italy; email: fuggetta@elet.polimi.it.

Permission to make digital/hard copy of part or all of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.

© 1997 ACM 1049-331X/97/0700-0283 \$03.50

General Terms: Human Factors, Languages, Management

Additional Key Words and Phrases: CASE, enabling technology, process-centered software engineering environments, process modeling languages, software process

1. INTRODUCTION

Process-centered software engineering environments (PSEEs) are becoming a reality. An increasing number of systems are showing up and are being applied to real software production processes. Existing products and prototypes are based on a variety of technologies and approaches, such as object-oriented languages and databases, state-oriented notations, rule-based languages, and logic languages (e.g., see Finkelstein et al. [1994]).

We believe that an important milestone has been reached in the development of this technology. There is some initial attempt to use it in industrial settings, and some feedback is being provided. It is therefore worthwhile to evaluate past experiences, to collect and organize feedback, and to provide a tentative roadmap for our research and experimentation activities. Moreover, it is essential to develop guidelines and methods to support this evaluation activity, in order to facilitate the collection and organization of information and their dissemination to practitioners and researchers.

This article aims at providing an initial assessment of the field. More specifically, it has two goals:

- (1) *To provide a roadmap for the evaluation and comparison of existing PSEEs:* The method we propose is based on an assessment grid that identifies the main constituents of the PSEE technology and their critical issues.
- (2) *To offer an initial assessment of the state of the art in the field:* This assessment exploits the grid to organize evaluations and comments on existing PSEEs consistently.

We will base this assessment on previous work (see, e.g., Fuggetta and Ghezzi [1994] and Lonchamp [1993]), on the evaluation of the PSEEs we developed in our own research activity, and on other relevant approaches developed in the research and industrial community. The three systems that will constitute the baseline for the evaluation are OIKOS, EPOS, and SPADE-1.

OIKOS [Montangero and Ambriola 1994] was started in 1989 in the framework of a research project on software engineering launched by the Italian National Research Council. OIKOS is the ancient Greek word for *home*, and it has been used in modern science as the root for words related to the environment, such as *ecology*. OIKOS is thus meant to be the “environment” for software development. The target of the OIKOS project is to ease the construction of PSEEs. OIKOS has been developed in Prolog and runs on a network of Sun workstations.

EPOS [Conradi et al. 1994b] has been developed at NTNU in Trondheim since 1989, first as part of a national project and later as a Ph.D. project. The emphasis is on flexible and evolving process assistance for software development and maintenance to multiple software developers. EPOS process models are expressed in SPELL, an object-oriented, concurrent, and reflexive modeling language. Process models expressed in SPELL are stored in EPOS-DB, a versioned software engineering database. The underlying platform is a network of Sun workstations, and Prolog is the main implementation language.

The SPADE project was started in 1991 to develop an environment for software process analysis, design, and enactment [Bandinelli et al. 1994c]. The main concept of the project is the adoption of extended Petri nets, augmented with specific object-oriented constructs to support product modeling. SPADE-1, the first implementation of the SPADE concept, is based on standard technologies that include a commercial object-oriented database (O₂) and tool integration facilities (DEC FUSE, Sun Tooltalk, and Microsoft OLE2). The last released version of SPADE-1 (version 3.0) was completed in the summer of 1995.

Clearly, we do not have the ambition to address in a single article and in a comprehensive and detailed way all of the issues and topics that are relevant to the field. Nonetheless, we believe that it is worthwhile to collect in a systematic way the available information, by exploiting some structured evaluation method. In this way, we provide some aids and a baseline for further assessment and dissemination activities and for the definition and evaluation of new research directions.

This article is organized as follows. Section 2 introduces the assessment grid. Section 3 discusses the modeling aspects of PSEEs, while Section 4 evaluates the architectural issues and implications. Section 5 presents some preliminary comments and evaluation on the application of PSEEs to real industrial cases. Section 6 presents related work. Section 7 tries to derive some “metalessons” concerning the use of the grid to assess the existing technology. Finally, Section 8 draws some conclusions.

2. THE ASSESSMENT GRID

The assessment will be accomplished using a simple comparison grid derived from several works on reference frameworks for software processes [Conradi et al. 1994a; Feiler and Humphrey 1993; Lonchamp 1993] and from the authors’ experiences.

2.1 Structure of the Grid

The grid, presented in Table I, is organized in three main parts.

2.1.1 PML Technology. Existing PSEEs are based on a variety of process-modeling languages (PMLs). To characterize them, we use the following categories:

Table I. The Assessment Grid

PML technology	Scope of coverage Linguistic paradigm(s) Modeling of process entities: —activities —products —tools —roles —organizations ... Modularity/composition/reuse Mechanisms for process enactment and evolution Cooperation and concurrency control Methodological support PML tool support
PSEE architecture	High-level architecture Interaction with production tools and tool integration facilities Behavior of the PSEE Data integration Inconsistency management and workspace synchronization Distribution and heterogeneity Support to measurement Efficiency of the implementation
Experiences	Reference examples Internal experiments Industrial projects

- (1) *Scope of coverage*: A PSEE and its PML can be adopted for a variety of reasons. A PML can be used to improve process comprehension and documentation through formal modeling, analysis, and simulation. This can help process improvement and process reengineering initiatives. A formal model can also be enacted to provide support and guidance to software developers. Therefore, a PSEE and its PML(s) can offer support to one or more phases of the *process lifecycle* (or metaprocess), that is, the set of activities that are pursued to improve a software process. We have identified the following main phases:¹

—*Requirements specification*: This phase aims at defining and/or documenting the requirements for the process, such as the expected performance and its general goals.

—*Assessment and problem elicitation*: This phase aims at (re)evaluating the performance of the process and at identifying its problems, according to the defined goals.

¹The sequential presentation we give of these phases does not imply that we advocate a traditional waterfall approach. It is just a suitable way to introduce these concepts. In real cases, different lifecycles can be introduced, according to the strategy and the goals of the organization. This is quite similar to software product lifecycles.

- (Re)design*: In this phase, the process is (re)designed (or reengineered) to address the requirements and needs elicited in the previous phase and to cope with the identified drawbacks.
 - Implementation*: In this phase, the designed or modified process is put into operation, by modifying models, procedures, organizations, policies, and, possibly, the process support.
 - Monitoring and data collection*: In this phase, qualitative and (when-ever possible) quantitative data on process performance are collected and analyzed to support further improvement initiatives (i.e., feedback on earlier phases).
- (2) Linguistic paradigm(s) used in the language (e.g., object-oriented, rule-based, state-oriented).
 - (3) Availability of constructs to model specific process entities such as products, activities, and roles.
 - (4) Linguistic constructs to support modularization, composition of specification, model configuration, and reuse.
 - (5) Mechanisms to support process model enactment (e.g., interpretation vs. compilation vs. mixed approaches, single thread vs. concurrent execution), and evolution (e.g., reflexive features).
 - (6) Linguistic constructs to model cooperation (among human agents) and concurrency control—we distinguish between synchronous (e.g., teleconference) and asynchronous cooperation (e.g., email).
 - (7) Methodological support to the different phases of the process lifecycle.
 - (8) Tool support for the PML (e.g., editors, simulators, impact analyzers, interpreters).

2.1.2 PSEE Architecture. The architecture of a PSEE can be characterized by the following aspects:

- (1) High-level architecture of the PSEE (i.e., main components of the PSEE and their integration and interaction strategy).
- (2) Basic mechanisms to integrate external production tools into the environment (e.g., tool wrappers).²
- (3) Behavior of the PSEE from the users' viewpoint (i.e., paradigm of the PSEE user interaction environment as perceived by software developers and by other process agents, e.g., project managers).
- (4) Mechanisms for data integration and structure/role of the repository in the PSEE architecture.

²In the remainder of the article, we use the term *tool integration facility* to refer to the FIELD technology, or similar approaches, and to related commercial products such as DEC FUSE, HP BMS, and Sun Tooltalk. In particular, in FIELD a tool is composed of a collection of services that can be invoked through a programmatic interface. A tool can broadcast messages and request the interleaved execution of services offered by other tools. For this reason, we refer to such tools as *service-based tools*.

- (5) Management of production workspaces (typically the Unix file system) where tools access their data; management of (possible) inconsistencies among workspaces and the PSEE database(s); and identification of operations accomplished outside the control of the PSEE (i.e., by directly calling a tool operating on some data in the workspace).
- (6) Distribution of the environment (e.g., centralized vs. client-server vs. peer-to-peer) and heterogeneous versus homogeneous architectures.
- (7) Support for the collection and evaluation of process and product metrics.
- (8) Efficiency of the implementation. Performance issues and adoption/integration of research prototypes to support the development of the PSEE (e.g., deductive DBMSs).

2.1.3 Experiences. The observations can be derived from practical experiences, classified as follows:

- (1) *Reference examples:* Applications of the PSEE/PML to reference examples in the literature (e.g., ISPW-6 and ISPW-7 examples).
- (2) *Internal experiments:* Internal applications of the PSEE/PML to examples and cases developed within the development team or derived from industrial projects.
- (3) *Industrial projects:* Applications of the PSEE/PML in industrial projects.

3. PML TECHNOLOGY

3.1 Scope of Coverage

OIKOS. In the initial manifesto, OIKOS was described as an environment to specify, design, and implement PSEEs [Ambriola et al. 1990]. Other goals were the comprehension and the documentation of software processes. Three factors influenced the project from the beginning: (1) the need for both an expressive specification language and a powerful concurrent enactment language, (2) the interest on the complex human interactions in process-centered environments, and (3) the decision to follow an experimental approach. OIKOS is based on two separate languages, Limbo and Patè, covering mainly the requirement specification and implementation phases, respectively.

EPOS. EPOS emphasizes expressing enactable and evolving process models, offering a spectrum of process support: from “soft” assistance to “hard” control and both manual and automated tool activation and execution. Nonenactable process requirements, specifications, and designs are not well supported, although some aspects of the PML can be used to describe more informal and incomplete models. Process models can be hierarchically specialized and dynamically refined at runtime, supported by an explicit metaprocess. There is marginal support for formal analysis or multiple views of process models. There is no formal metrics support.

The process agent is assumed to have a good knowledge of the process model and its PML, as the external view is very close to the PML semantics.

SPADE-1. The long-term goal of the SPADE project is to provide methods and techniques to support the entire process lifecycle. The technology that has been developed so far, however, is mainly oriented toward supporting the implementation and monitoring phases.

SLANG (the SPADE Language) can also be used to support analysis and design, because its graphical syntax (based on traditional Petri nets) is easy to learn and to use. However, in these phases several features of the language that are mainly oriented toward supporting enactment are useless, while other high-level concepts useful for supporting these earlier phases are missing (as discussed later).

Other Systems. Most approaches that have been developed so far basically address implementation and monitoring, as in APPL/A [Sutton et al. 1995] and in the Japanese project Hakinowa [Iida et al. 1994].

In some cases, there has been a significant attempt to enlarge the scope of coverage. For instance, MERLIN offers (1) statecharts and ER diagrams as high-level modeling aids and (2) a Prolog-based language to support process implementation [Junkermann et al. 1994]. In PADM, there is a specification language called BM (based on temporal logic), while implementation is supported by PML (Process Management Language³), which offers such features as subtyping, persistency, reflection, and concurrent threads of execution [Bruynooghe et al. 1994]. Different languages for different phases of the process lifecycle are also provided by SMART [Garg et al. 1994].

Evaluation. Based on our experience in using OIKOS, EPOS, and SPADE-1, a first important issue is the distinction between the different phases of the process lifecycle. Their goals are different, and therefore their outcomes differ significantly.

During requirement specification (process analysis), the goals are mainly to understand the process, to make it explicit, and to facilitate its communication, comprehension, and assessment. The resulting model will describe the expected and desired behavior of the process, the roles to be played, and the high-level procedures to follow.

Conversely, the implementation phase aims at producing an automated process support that facilitates the achievements of the objectives and goals stated in the requirement phase. The result of this phase is often a complex “code” that entails many details such as tool integration and process step encoding. Most of these details are almost irrelevant during analysis and design. For an example, we can consider some experiences gained in

³In PADM, PML stands for Process Management Language, and it is the name of the language used to model processes. It must not be confused with the acronym PML, used in this article generally to indicate a Process Modeling Language.

applying SLANG to describe an industrial process [Bandinelli et al. 1995]. The high-level description of the process that was generated with the purpose of improving the quality manual is quite different in terms of structure and content from its enactable model used to provide process support, even if we used the same language [Bandinelli et al. 1996b]. Language constructs are used in different ways. For instance, the activity construct of SLANG, when used to describe the process, was exploited to identify at different levels of granularity the logical steps of the process. In the enactable model, activities are used to modularize the “code” used to control tools or to “implement” specific cooperation patterns. Thus, in the enactable model, SLANG activities do not express the logical steps of the process. They are just used as the modularization facilities of traditional programming languages (e.g., procedures, coroutines, and tasks). It is, therefore, important to emphasize the basic distinction between *specification of requirements* (i.e., *the problem*) and *specification of the implementation* (i.e., *the solution*).

Consequently, a critical issue is to understand the characteristics of the PMLs that are needed to support the different phases of the lifecycle. In the earlier phases, the requirements for the PML are basically expressiveness and support for analysis, while in the implementation phase, the emphasis is on flexible mechanisms to build an effective and efficient process support. For this reason, we introduce a more precise and detailed characterization of PMLs based on the phases they support:

- (1) We will use the term *PML* to indicate any language that can be used to provide some description (or “model”) of the software process.
- (2) PMLs can then be classified according to the phase of the process lifecycle they support and the level of abstraction they provide. We distinguish three different types of PMLs:
 - Process specification languages (PSLs)*: These languages are suited to support the requirement specification and assessment phases.
 - Process design languages (PDLs)*: These languages offer features that are useful in the design phase.
 - Process implementation languages (PILs)*: These languages are mainly used in the implementation and monitoring phases.⁴

Notice that the distinction we propose does not necessarily require the existence of separate languages. As we discuss in more detail in the next section, there are at least two extreme strategies: (1) one large PML that includes all of the features that are typical of PSLs, PDLs, and PILs and (2) distinct and separate languages for the different phases of the process lifecycle. Here we want to stress that *most of the problems we encounter in understanding and comparing existing PSEEs and PMLs stem from an unclear identification of the role of the PML with respect to the different phases of the process lifecycle* (this will be discussed later in the article).

⁴The assessment and problem elicitation phase can, in general, take advantage of process models produced at different levels of abstraction using different types of PMLs.

3.2 Basic Characteristics of a PML

In this section we provide a discussion of the three main aspects that characterize a PML: (1) linguistic paradigm(s), (2) modeling of process entities, and (3) features to support modularity, composition, and reuse.

3.2.1 *Linguistic Paradigm(s)*

OIKOS. Limbo, the OIKOS specification language, is based on tuple spaces, coordinators, and angels. A *tuple space* is a passive container of tuples that represent the state of a computation. A *coordinator* is an active entity that operates on a tuple space by reading and writing tuples from it. A coordinator can also send tuples to other tuple spaces. *Angels* are the basic process entities, specified by their external behavior (the messages they can send) and constrained by extended path expressions. According to the OIKOS lifecycle, a process is first specified by angels and then refined, replacing the angels with tuple spaces and coordinators.

Patè is a concurrent language based on hierarchical multiple tuple spaces and rule-driven agents. As in Limbo, a tuple space is a passive container of tuples that can be read and written by the agents that are active on the tuple space. An agent is defined by a set of forward rules and a path expression. A rule is fired when it is enabled by the path expression and when its precondition is satisfied by the tuple space (i.e., it unifies with a subset of the tuple space). The precondition of a firing rule can remove tuples from the tuple space. The body of a rule is a Prolog goal without side effects. The postcondition defines the tuples to be written in the agent tuple space and in other tuple spaces whose addresses are known to the agent.

The rationale of this choice is the intrinsic reactive nature of the software process and its structural complexity. The specification language has been designed with a refinement strategy in mind; the enactment language is well suited to modeling reactive concurrent systems.

EPOS. SPELL is a concurrent and reflexive language that has Prolog as a subset. It supports object-oriented modeling (ERA data model with triggers). Process entities are described by SPELL types and instances of them. Activity (task) networks in SPELL can express both goal-oriented process models, using static rules and constraints for automatic network planning, as well as activity-oriented process models, using dynamic pre- and postconditions and scripts. The metaprocess for defining and evolving the process model can be expressed using the reflective features of SPELL (similar to metatypes in Smalltalk).

The reasons for having Prolog as a base language are the classic ones: interpretation to support evolving models, reasoning facilities (e.g., on pre- and postconditions by the Planner tool), easy creation of mockups for process models and databases, and access to a powerful user interface tool (PWI). In some sense, SPELL is Prolog extended with object orientation (types), reflection (metatypes), and tasking.

SPADE-1. SLANG is based on ER nets, a high-level extension of Petri nets. In SLANG, tokens are associated with values, and transitions are associated with rules. Every token is implemented and accessed as an object in an object-oriented database. Every rule is composed of a condition, used to extract an enabling tuple (i.e., tokens satisfying the condition) from the database, and a set of statements to specify how to create the output tuple. Thus, SLANG integrates object-oriented techniques for product modeling, with Petri nets for (strict) process modeling. Moreover, every transition is associated with a forward rule (“if there is at least one enabling tuple, the action part can be executed using any one of the enabling tuples as input”). The rationale of this choice can be summarized as follows:

- The artifacts produced during software development are complex and strictly interrelated. The object-oriented paradigm makes it possible to easily describe both the structure of artifacts and their relationships.
- A software development process is composed of multiple activities that are executed in parallel and that need to be synchronized. The extended Petri net notation adopted in SLANG makes the modeling of these aspects quite natural and easy to understand.

Other Systems. There have been several attempts to identify a reasonable classification of PML paradigms. For instance, Curtis et al. [1992] identify five different paradigms:

- (1) Programming models (e.g., APPL/A and PADM/PML).
- (2) Functional models (e.g., HFSP [Katayama 1989]).
- (3) Petri net (and state transition) models (e.g., SPADE-1, Melmac [Gruhn and Jegelka 1992], and ProcessWeaver [Fernström 1993]).
- (4) Plan-based models (e.g., GRAPPLE [Huff and Lesser 1989]).
- (5) Quantitative models (e.g., system dynamics [Abdel-Hamid and Madnick 1991]).

Another classification is proposed by Lonchamp [1994], where six different classes are identified: graphical, net oriented, procedural, object oriented, rule based, and multiparadigms. In general, the issue is still debated in the community (see Section 3.2.4).

3.2.2 Modeling of Process Entities

OIKOS. Typical entities used in process modeling are process,⁵ office, environment, desk, role, and service. Process modeling is performed top-down, by refining an entity in terms of its depending entities. For instance, a process is recursively composed of an office, some services, and some processes. The leaves of the refinement tree are roles and services.

⁵Here, an OIKOS “process” is an entity that combines managerial and technical aspects of software development.

Services [Ambriola et al. 1994] are in charge of managing resources such as documents and workspaces. Documents are stored in repositories and manipulated by tools that operate in workspaces (i.e., Unix directories).

Entities are used in Limbo to specify the process. The refinement of a Limbo specification produces a Patè program. Artifacts are modeled in the database by using a simple DDL (data definition language) that allows the definition of a hierarchy of document types.

EPOS. Products, activities (tasks), and other process elements are modeled as SPELL types and their instances. This can occur at different levels of granularity.

Tasks have predefined properties, allowing them to stand in a task network with both a hierarchical (decomposition) and horizontal (chaining) dimension. A task type has also static pre- and postconditions for static reasoning, and dynamic pre- and postconditions and a script for dynamic triggering and execution. Task types contain formal parameters and formal decompositions, that is, the legal set of input-output operands and of subtasks, respectively. This expresses a minimal graph grammar or template to describe network fragments.

A predefined product model (i.e., SPELL types to create product “families”) is supported by special product/workspace tools outside the basic process tools. Production tools are enveloped and modeled as task types, with a script to activate the tool and an extra input parameter to receive feedback. A mailbox facility with special “product” types is available to send and receive messages.

A task can be associated with a role and then executed by agents playing that role. Projects are special tasks, containing change requests and logs, product/versioning information, available resources, and communication protocols. A project task is associated with a database transaction, and the project and transaction hierarchies map directly onto each other. Concepts and associated formalisms for modeling versioning, transactions, external workspaces, and organizations are expressed outside of SPELL.

SPADE-1. SLANG built-in features are oriented toward model activities and products. It is also possible to integrate and control different kinds of tools at two levels of granularity (activation/termination of a tool and invocation of individual tool services). SLANG does not include a built-in construct to describe roles, goals, and workspaces. A primitive SLANG construct for roles would be useful to support earlier phases in the process lifecycle. At this time, roles can be implemented as objects (i.e., tokens) and associated operations (i.e., object methods and transition rules) in a SLANG activity. Workspaces are explicitly managed by specifying in some SLANG activity the operations needed for their creation and manipulation. As discussed in more detail later on, the approach adopted in SLANG is to have a simple language where process-specific concepts are modeled explicitly by the process modeler.

Other Systems. Some existing PSEEs incorporate process-specific concepts as predefined language constructs. For instance, PEACE offers specific constructs to represent tasks, artifacts, agents, and roles [Arbaoui and Oquendo 1994]. In other cases, the language includes only generic constructs, with little, if any, process-specific semantics.

3.2.3 *Modularity, Composition, and Reuse*

OIKOS. The modular entity of Limbo is the angel. Angels are composed to form complex systems. In Patè, the modular entities are the tuple space and the agents.

EPOS. Most SPELL properties can be declared private or public in an ADT style. There is no explicit submodel or subschema construct, except for simple decomposition and chaining definitions (type-level constraints or templates; see the next point). Process models are defined in a (sub)project context, corresponding to versioned (sub)transactions. Thus, it is possible to “inherit” a template model from the parent project, which then can be customized and evolved locally. Project-internal tasks can be decomposed in hierarchies, representing submodels at different abstraction levels (e.g., composite vs. basic activities).

SPADE-1. SLANG offers a modularization facility that makes it possible to structure a process model as a hierarchy of SLANG nets. The modularization construct is called an *activity*.⁶ Each activity is a SLANG net with an interface and an implementation. The implementation of an activity can include the invocation of other activities. Each invocation implies the creation of a new instance (active copy) of the invoked activity, which is executed asynchronously with respect to the caller.

Other Systems. It is not easy to classify the modularization facilities offered by existing PMLs. Obviously, the modularization facility of each PML is often implicitly defined by the paradigm of the language used.⁷ In most cases, they are just missing, that is, the process model is conceived as a monolithic specification. A significant example of a language that has a rich set of constructs to support modularization is APPL/A, where the features of Ada are fully exploited to organize and structure a complex process specification in a set of cooperating tasks and packages.

3.2.4 *Evaluation of the Basic Characteristics of a PML.* As discussed in Section 2, real-world software production processes can be very large and can have a variety of process entities that must be modeled and supported, perhaps with different granularities of precision. Typical entities to be modeled are activities, products, versioning, and configuration policies, and user, roles, organizations, projects, tools, transaction models, workspace

⁶Here, a SLANG “activity” is just the keyword adopted to identify a net fragment.

⁷This is not a general situation. For instance, SLANG is based on Petri nets, which intrinsically do not offer any modularization facilities.

management policies, and user interface metaphors and styles of interaction.

Key issues are *how to design the PMLs that support the process lifecycle and to what extent they have to include constructs to directly support the modeling of all or some of the above entities*. The chosen strategy for PML design will influence the size and complexity of the PMLs and the resulting process models themselves. That is, what is the borderline between the fixed or “core” constructs of a PML, and the soft or declarable policies built on top of it?

To illustrate the PML design problem, we present some examples concerning the development of a PIL:

- (1) *Tool integration*: Many PILs provide a synchronous mechanism to model the invocation of external tools (e.g., a compiler). This mechanism allows the process modeler to specify initial parameter preparation, actual tool invocation, and final control operations.

Other PSEEs offer lower-level constructs to invoke and control tools. For instance, OIKOS, SPADE-1, and EPOS include an asynchronous construct to start an external tool and a triggering construct to detect events occurring outside the control of the PSEE. These lower-level constructs are less abstract and “readable” than the synchronous constructs discussed above. Thus, to achieve the same effect of other more abstract constructs, it is necessary to properly combine these lower-level constructs in a process model fragment that describes how to invoke and control the tool. However, these lower-level constructs are essential to manage service-based tools. To manage the interaction with these tools, it is necessary to specify reactive policies to each tool operation (e.g., an editor file-open request), and even the policy to control mutual tool interaction (e.g., automatic invocation of the editor when the compiler identifies an error). Thus, by adopting these lower-level constructs we lose on expressiveness and readability, but we gain on flexibility and degree of control.

- (2) *Support for evolution*: Some PSEEs offer high-level features to support evolution, through predefined PML constructs. They define “the” policy(ies) to evolve the process model. For instance, in MELMAC it is possible to invoke a command that allows the modification of a process model fragment immediately before it is enacted.

Other PILs (e.g., EPOS and SPADE-1) offer basic, reflexive constructs to support the specification of the “process of change” as part of the process itself. The actual evolution policy must therefore be explicitly “programmed” by the process modeler. In this way, the policy can be freely changed or adapted according to specific process requirements. A reflexive and rather low-level PIL is therefore useful to “implement” an evolving process support. Such a PIL will be, however, unsuitable to express in a very readable way more high-level policies and strategies to accomplish process changes in a corporate organization.

These two examples illustrate that PSEEs often have embedded improper features as basic PIL constructs. PILs have often been enriched with high-level features, because the (implicit) goal was to use them as PSLs or PDLs as well.

The key issue is therefore how we design and evaluate the effectiveness of a given PML (or set of PMLs). According to our experience, it is possible to identify some design strategies for the development of PMLs.

A PSEE can be based on any of the following approaches:

- (1) There is just one PML to cover all of the different phases of the process lifecycle. This PML contains predefined language constructs to support all of the phases in the process lifecycle, that is, a single and probably very large language that includes all process-related concepts (e.g., roles, organizations, goals, and evolution facilities). This is the approach implicitly adopted by most existing projects, even if the emphasis is in many cases on the implementation phase.
- (2) There are two or more PMLs with different characteristics to address different phases of the lifecycle. This is the approach adopted by OIKOS, where Limbo plays the role of PSL/PDL and Patè is the PIL, and by other systems such as PADM and MERLIN. We believe this is the approach to be followed to reduce the complexity of PMLs and to better support the different phases of the process lifecycle.

More detailed requirements for each class of PMLs can be summarized as follows:

PSLs. These are quite similar to the languages used for business process modeling, conceptual modeling, and requirements elicitation. A language to support this early phase of the process lifecycle should provide, for instance, the concepts of role, organization/group, business rules, flows of information, and time constraints. This is consistent with our experience (e.g., [Bandinelli et al. 1995]).

In general, PSLs should be formally defined in order to analyze process specifications, to prove properties or evaluate performance, and to simulate/prototype the process. It must be possible, however, also to use these languages in an informal way at the beginning of the specification activity, as experiences in traditional software development have demonstrated (see, e.g., [Fuggetta et al. 1993]).

PDLs. At the design level, we need languages to specify or express the architecture of the enactable process model. In particular, we should be able to reuse concepts from software architecture. Here, the emphasis is on constructs to specify modules, concurrent processing elements, module connectors, etc. [Garlan and Shaw 1993]. Actually, an enactable process model is a complex and structured entity that is quite similar to traditional software. APPL/A is a PIL that also offers advanced PDL facilities that have been directly inherited from Ada (e.g., packages and tasks).

PILs. The experiences from the implementation level are more mature. It is therefore possible to offer a richer and more detailed discussion of the ideal features of a PIL.

First, it is possible to identify three general strategies for designing a PIL:

- (1) The PIL contains a rich set of process-specific concepts. This is the choice of several languages, such as PEACE [Rabbi and Oquendo 1994].
- (2) The PIL contains one set of very basic constructs that are used by the process implementor to build more complex concepts. The basic assumption here is that, without enough knowledge and experience on the characteristics of PILs, it may be sensible to let certain entities be definable in a very flexible way. For instance, the PIL may not provide the concept of “role” as predefined and may not offer basic type/object constructors to model such an entity. In this class, we have SLANG and Patè.
- (3) Different process entities are covered by separate sub-PILs, but are wholly independent of and orthogonal to the “core” PIL. For instance, organizations and products are modeled using different languages. Again, the rationale is not to freeze into the “core” PIL those concepts that can change from process to process. In this class, we have EPOS. This might be considered a generalization of the bilevel approach advocated by Kaiser et al. [1993]. Clearly, this strategy introduces the critical issue of how to manage the interoperation among different languages [Montangero 1995].

According to our experience, the features that must be offered by a PIL are as follows:

- (1) *Modeling of concurrent activities:* This requirement derives from the nature of software processes that are typically concurrent activities. In our PSEEs, this is accomplished using Petri nets (SLANG), concurrent agents (OIKOS), and cooperating transactions and their internal tasks (EPOS). This requirement has an important impact on the enacting mechanisms that have to be provided (i.e., the PIL run-time support).
- (2) *Modeling of products:* We advocate an object-oriented paradigm, since in our experience this seems to be the paradigm that is more appropriate to describe products and process artifacts. However, use of object-oriented technology (such as OODBMSs) is still problematic (we discuss standardization of technologies for the PSEE database later in the article).
- (3) *Integration mechanisms:* Mechanisms to control at a fine-grained level (e.g., messages) the tools used by PSEE users are essential to manage the interaction among users and between users and the PSEE in an effective and consistent way [Bandinelli et al. 1996b].
- (4) *Modularization facilities:* This is needed to support independent development, enactment, and maintenance of different process model fragments.

- (5) *Basic mechanisms to support workspace management*: Examples are features to describe the structure of a workspace and the policies that have to be followed to manage it.
- (6) *Basic mechanisms to describe concurrency control*: The PIL should offer very basic mechanisms to specify the different policies for concurrency control. Most of the systems supporting so-called long-lived transactions offer concepts that are too high level, which de facto implement predefined policies. They are not general enough to model and support the variety of cooperation patterns being used in software processes.
- (7) *Reflexive features*: This is a key feature that supports evolution of the process model and/or its enactment state, either in advance or on-the-fly.

As a summary of our discussion on PILs, we would like to reflect on the way these classes of languages are designed and implemented. Based on our experience, we claim that, for a PIL to cover the above list of requirements, the actual choice of paradigm (e.g., rule-based vs. state-oriented) is not so important. Actually, most PILs are hybrid languages that integrate different paradigms. For instance, SLANG integrates Petri nets, object-oriented modeling of products, forward rules to specify transition actions, and the activity construct (similar to multiple-entries, multiexits coroutines) to support modularization. What really counts are the following:

- Overall PIL enactment architecture*: Critical issues are the integration with tools, the parallelism and efficiency of the PIL run-time support, and its degree of distribution.
- User-PSEE cooperation paradigm and views*: Does the software developer have to know the actual PIL and the internal process model expressed by it? The main goal of the PSEE should be to give flexible enactment support at the appropriate level (e.g., control, automation, guidance, reasoning, explaining). The user must then be presented a comprehensible view of his or her current work context, with proper connections to coworkers' activities. Given this, the user can (more) intelligently execute and relate to his or her own role in the overall process. This does not only depend on the PIL paradigm and can even be completely decoupled from it (see, also, the next section) [Bandinelli et al. 1994a].
- Easy user-level evolution of the process model*: Again, the goal is to provide an understandable view of the model, so that it can be changed even by the process agents themselves, if and when needed.

3.3 Mechanisms for Process Enactment and Evolution

OIKOS. A Patè program is executed by a distributed interpreter fully integrated in the OIKOS architecture. Tuple spaces and agents can be added dynamically to a running system. To start the execution of an agent, the interpreter loads the agent rules from a repository (the file system, in

the current implementation). This mechanism allows a low-level degree of evolution.

The Patè interpreter is implemented in Prolog, augmented with a library for network communication. The interpreter is a concurrent program made of a collection of Unix processes allocated over the network. Many instances of the Patè interpreter can be active at the same time. Each instance of the Patè interpreter runs a hierarchy of tuple spaces.

EPOS. Each project/transaction has its own instance of the Prolog interpreter. Tasks are executed in a coroutine fashion within a transaction; that is, only one modeled task can be active at one time. However, such an active task will only run for a short time before it (re)initiates some external user/tool via a BMS. It then passivates itself to wait for responses from the user/tool, and another task can therefore soon be run. Tasks with satisfied dynamic preconditions are either run automatically or presented to users as parts of an agenda. The evaluation of preconditions is performed automatically, but can have side effects, for example, upon reading a mailbox. True parallelism occurs between projects/transactions and between invocations of external production tools.

The process model is versioned and reflective. It can modify itself during its execution, modeled by special SPELL metatypes and instances thereof, and through incremental task network construction.

SPADE-1. Every instance of a SLANG activity (i.e., an active copy) is executed by an independent SLANG interpreter. Thus, during enactment, multiple active copies are concurrently executed by many SLANG interpreters. Different SLANG interpreters can exchange information through shared places, that is, Petri net places that are shared by different SLANG activities. Consistency (i.e., atomicity of Petri net transition) is always ensured because a SLANG transition is executed as an ACID transaction.

Evolution is supported through reflexive features (late binding, dynamic invocation, and visibility of programs as data). In particular, activity definitions (i.e., their graphical representation) and active copies' states (i.e., the state of enacting instances of activity definitions) can be seen as tokens and, therefore, can be manipulated by some SLANG activity as any other token.

Other Systems. APPL/A is a traditional compiled language. There is no process engine (e.g., interpreter) because each APPL/A "code" is compiled and linked to create an autonomous executable program. The same approach is adopted in Process Weaver, even if applied within the context of a completely different language (i.e., based on Petri nets).

Reflection appears to be the chosen feature to support evolution. In other cases, ad hoc mechanisms are provided in the environment to support changes to the process model while it is enacted. MELMAC offers an intermediate solution where it is possible to stop enactment before executing a net fragment and to support its on-line customization. Finally, in other systems, process models are supposed to evolve off-line with respect

to their enactment and support to the software development process (e.g., APPL/A).

Evaluation. As in traditional software development, process model execution can be achieved by exploiting two basic approaches:

- (1) compiled approaches (e.g., APPL/A); and
- (2) interpreted approaches using an interpreter or “process engine” (as in most existing systems).

The alternative followed by most researchers (i.e., the adoption of the interpreted approach) is certainly due to the assumption that the technique can be exploited to effectively support process model evolution during enactment.

3.4 Cooperation and Concurrency Control

OIKOS. OIKOS does not provide any specific support to synchronous cooperation. Asynchronous cooperation is supported by controlling the behavior and interaction of tools used by software developers.

As for concurrency control, roles cooperate inside the same process by exchanging messages and by synchronizing on document access. At a lower level, concurrency is obtained by using different Patè agents that perform independent actions. These agents synchronize themselves by accessing the tuple space, where they expect to find specific tuples.

EPOS. The underlying EPOS-DB offers versioned, long, nested, and cooperating (nonserializable) transactions. Similar cooperative transaction models are found, for example, in Korth et al. [1985], Fernandez and Zdonik [1989], and Skarra [1989]. An external, file-based workspace is associated to a database (sub)transaction. Both product- and file-based check-out/check-in operations are available via the EPOS Workspace Manager and its ECM tools. Synchrony between the external workspace and the transaction database is ensured by extra scripts for tool activation and/or by local, programmer-controlled check-ins.

A Workspace Unit Declaration Language (WUDL) is used to specify a given product and its version. It can also express lock modes (read-sets and hard or soft write-sets) and cooperation protocols. The latter include notification modes (busy, lazy), degree of sharing (copy-on-write, copy-on-read, common), workspace directory structures, and consistency predicates. WUDL and SPELL are capable of expressing flexible, user-defined policies for handling access conflicts (e.g., breaking of locks, managing version skews and associated merging, reestablishing consistency after changes, and controlling change propagation). The goal is to keep cooperating and affected partners notified about product status and possible conflicts.

A WUDL specification is interpreted by a Cooperation Manager (CHAT tool) and by associated watchdog tasks in SPELL. The WUDL is also used by a Transaction Manager to help the human project manager and developer to partition, schedule, and further organize the proposed work and its

transaction structure. For instance, it will accomplish impact analysis of proposed changes. A simple product browser is also available.

SPADE-1. Cooperation at the user interaction level can be modeled in SLANG by properly controlling and influencing the behavior of tools used by process agents. This is achieved by exploiting black transitions to send messages to tools, and user interface places to trap and reify the messages generated by tools (for more details, see Section 4 and Bandinelli et al. [1996b]).

Every SLANG transition is executed atomically as an ACID transaction operating on the O_2 database. This provides the basic mechanisms to support concurrency control. A SLANG activity combining more transitions (i.e., more transactions) can then be used to implement complex cooperation patterns (e.g., nesting transactions, chained transaction, sagas [Bandinelli et al. 1996a]).

Other Systems. Most approaches provide built-in facilities to model concurrency control, that is, long-transaction mechanisms (see, e.g., Marvel, MERLIN, and Adele). As for supporting cooperation among process agents, Oz integrates facilities to support synchronous cooperation between remote users [Ben-Shaul and Kaiser 1994]. Most PSEEs, in general, make it possible to support only asynchronous cooperation.

Evaluation. The issue of supporting cooperation has not been specifically addressed in most systems. The only notable exception is Oz. Some experimentation has also been conducted in the SPADE project [Bandinelli et al. 1996b]. These approaches emphasize the role of the PSEE as the executor of the cooperation policy and as the infrastructure to support asynchronous cooperation. Synchronous cooperation (e.g., real-time conferences) is delegated to external CSCW components controlled by the PSEE through its tool integration facilities.

As for concurrency control, the main issue is once again long-lived transactions. Do we need this kind of construct? Should we include it in the PIL ad hoc mechanisms and constructs (as in EPOS), or should we go with simpler languages where it is possible to support different cooperation patterns by explicitly programming them (as in SLANG)? We do not have a definite answer yet.

3.5 Methodological Support

OIKOS. The development of a software process is a sequence of refinements that substitute an abstract entity (an angel) with a more concrete set of entities. This process is supported by a structured graphical editor that controls the correctness of the refinement steps and that integrates the graphical and the textual views of the process to be modeled.

EPOS. SPELL is a reflexive language, and an EPOS process model is versioned according to the EPOS-DB. The operands of a SPELL task can be any process model fragment. EPOS thus allows full specification of the metaprocess for constructing and maintaining process models.

During enactment, both the task network and its types can be changed through specific tools. After enactment, the local subproject is closed, and its new or revised types are committed to the parent project.

SPADE-1. SPADE-1 does not provide any predefined method to support the development and evolution of process models. The metaprocess, whatever it is, can be modeled as part of the process using the reflexive features of SLANG. Some experiences with the usage of SLANG are discussed in Bandinelli et al. [1995]. This work suggests an iterative approach to the specification of the process that is based on the repetition of three basic steps: knowledge acquisition (i.e., What are the basic process characteristics?), formalization (i.e., representation of the process knowledge using SLANG), and validation (i.e., assessment of the model against the original requirements and inputs).

Other Systems. A method for process development and evolution is the Prism methodology [Madhavji and Schäfer 1991]. It emphasizes the relevance of change and evolution, and the need for keeping track of the rationale for such changes. PADM/PML offers a metaprocess (PMMS [Snowdon 1992]) to construct and evolve process models.

Evaluation. This is an extremely critical point that is directly related to the experiences in using PMLs and PSEEs. Most of the technology that has been developed so far does not include any methodological support. We have languages and tools, but very few methods to guide the development of a process model and its enactment, extension, and evaluation. The analysis of software processes is often accomplished without any systematic approach. There is little if any support to guide the selection of the portion of the process that should be modeled and supported first.

3.6 PML Tool Support

OIKOS. Besides the Patè interpreter, a graphical tool is available to edit and browse Limbo specifications. This tool provides support and assistance in the refinement of Limbo specifications into Patè programs.

EPOS. The most important EPOS-PM tool is the Process Engine on top of the SPELL Interpreter. This enacts runnable tasks and interprets the SPELL types. It also displays the process model under execution and its agenda to the user. The Process Engine is connected to a tool integration facility (HP BMS) for communication with external production tools. A task network (re)Planner will incrementally (re)construct lower-level subtasks to expand a goal-oriented and incomplete parent task. The Planner works in close cooperation with the Process Engine.

A Schema Manager with a type editor/browser is used to evolve the types. More high-level and project-related tasks must be planned by a human project manager and interactively generated by the EPOS Project Manager, consisting of a Task Network Editor and a Transaction Manager. The latter will set up the WUDL descriptions (see below) and the version descriptions to regulate reads/writes of the EPOS-DB.

A Workspace Manager is in charge of external products and of transfer of files by our ECM check-in/check-out tools, interpreting WUDL descriptions. The workspace is created upon start and is flushed at end, and task types contain scripts for continuously checking in changed files to keep the client's internal process model and external workspace consistent. A Cooperation Manager (CHAT) sets up and interprets cooperation protocols between transactions described as project tasks.

SPADE-1. SPADE-1 includes a SLANG graphical editor, a SLANG compiler and type checker, a tool integration facility, a process monitor to display the state of process enactment, two prototypes of a user agenda, and bridges to integrate DEC FUSE, Sun Tooltalk, and OLE2 tools. It also includes the SLANG interpreter and the SPADE manager (controlling start-up and shut-down operations).

Other Systems. The PML tool support offered by most PSEEs is similar to what is currently provided by OIKOS, EPOS, and SPADE-1. Some environments also offer some more innovative features. For instance, Articulator offers tools supporting knowledge acquisition and process simulation [Mi and Scacchi 1990].

Evaluation. All systems have editors to build and maintain process models and to enact them. The latter implies a concurrent/distributed run-time environment to support real, nontoy processes. However, there is often little support for visualization, viewing, explanation, simulation, etc. These kinds of issues have not been addressed in detail. In particular, we need to clarify how the different components or languages of a PSEE should be designed and interoperated to take care of (parts of) this task. For instance, the visualization of the process state should be performed taking into account a high-level view of the process such as the one offered by a specification written in some PSL. Conversely, at run time we execute a PIL "program" that, in general, provides a much less abstract view of the process. Therefore, we should be able to develop reverse mapping tools to relate the state of enactment to the more abstract specification of the process. We experienced these phenomena in all of our systems (OIKOS, EPOS, and SPADE-1).

4. PSEE ARCHITECTURE

4.1 High-Level Architecture

OIKOS. OIKOS architecture is based on a virtual machine, Expo 2.0, that allows execution of Patè programs, integration of external tools, and interaction with the users through a graphical interface (see Figure 1). In Expo 2.0, new components can be connected to the machine. The interaction with other components is performed through message passing. Unlike other PSEEs, Expo 2.0 does not exploit a commercial tool integration facility and is based on a custom communication protocol.

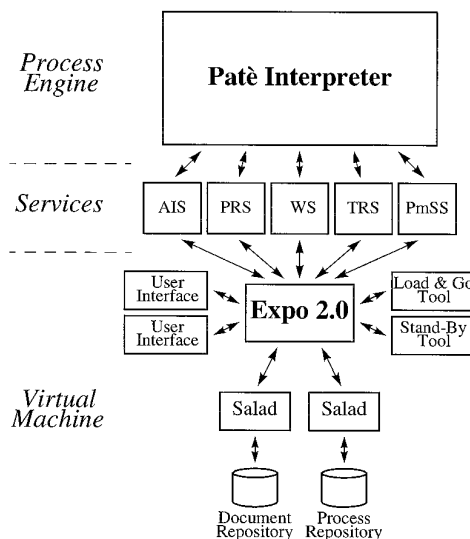


Fig. 1. OIKOS architecture.

On top of Expo 2.0, a layer of services is in charge of managing documents and tools. This layer exploits a centralized database implemented using Salad, a logical deductive DBMS [Chimenti et al. 1990].

EPOS. The EPOS architecture is depicted in Figure 2. It is composed of four layers:

- (1) *An underlying EPOS-DB built upon the C-ISAM index-sequential file system:* EPOS-DB is a proprietary, client-server database that stores process models in the context of versioned, nested, long, and cooperating transactions. It has a structurally object-oriented data model and its own change-oriented version (COV) model.
- (2) *Enactable process models cached at the client sites:* These are accessed using the DDL/DML (Data Definition Language and Data Manipulation Language) of the underlying EPOS-DB. These process models are essentially task networks, whose customized types are interpreted by EPOS-PM tools. There are also some non-*SPELL* descriptions, particularly *WUDL*, to describe versioning, transactions, workspaces, and cooperation protocols.
- (3) *A *SPELL* Interpreter (EPOS-PM) at the client sites:* This adds full object orientation and reflection to the basic data objects accessed and cached from the EPOS-DB.
- (4) *Process tools (EPOS-PM) that manipulate the process model to support its creation, modification, and enactment:* See Section 3.6. These tools comprise
 - the Process Engine (for tasking),
 - the (re)Planner,

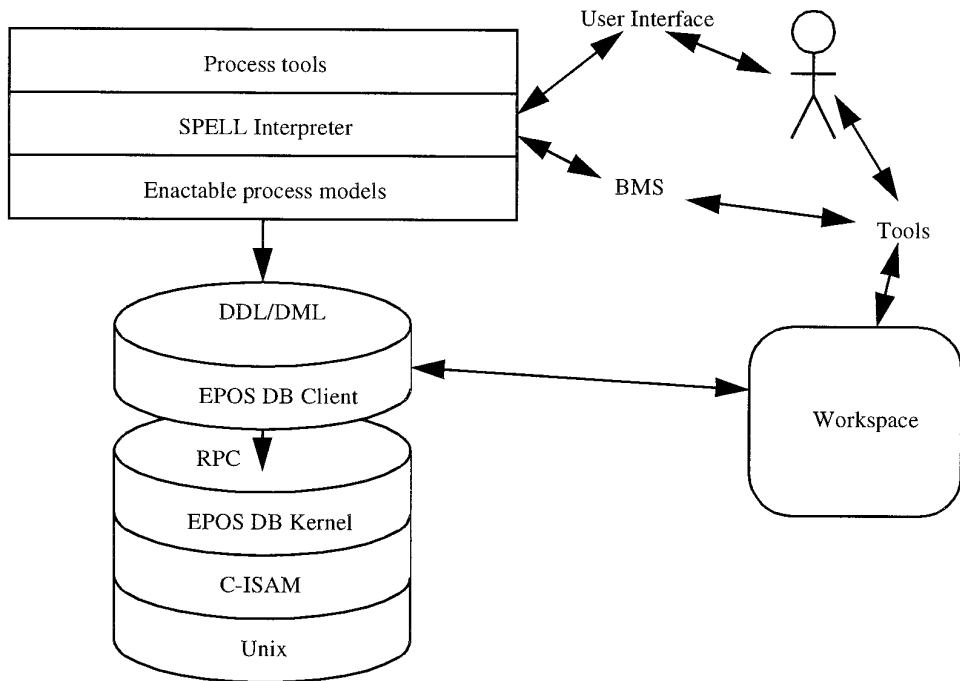


Fig. 2. EPOS architecture.

- the Schema Manager,
- the Project Manager (with a Task Network Manager and a Transaction Manager),
- the Workspace Manager, and
- the Cooperation Manager called (using WUDL).

SPADE-1. SPADE-1 is based on a three-layer architecture (see Figure 3). The process enactment environment includes the SLANG interpreters that are implemented as threads of Unix processes called *process engines*. Each process engine is an O₂ client. The user interaction environment includes the tools used by the SPADE-1 user to accomplish software development activities. This layer also includes a filter that is used to trigger events in the user interaction environment and to make them visible as tokens in some of the SLANG activities being executed. Similarly, the filter can trigger service requests issued by a SLANG interpreter and forward them to the proper tools in the user interaction environment. Basically, SPADE-1 behaves like a *reactive machine*, which reacts to user-generated events.⁸ Finally, the third layer includes the SPADE repository, which is implemented on top of the O₂ OODBMS.

⁸This does not mean that the process enactment environment cannot start an operation autonomously.

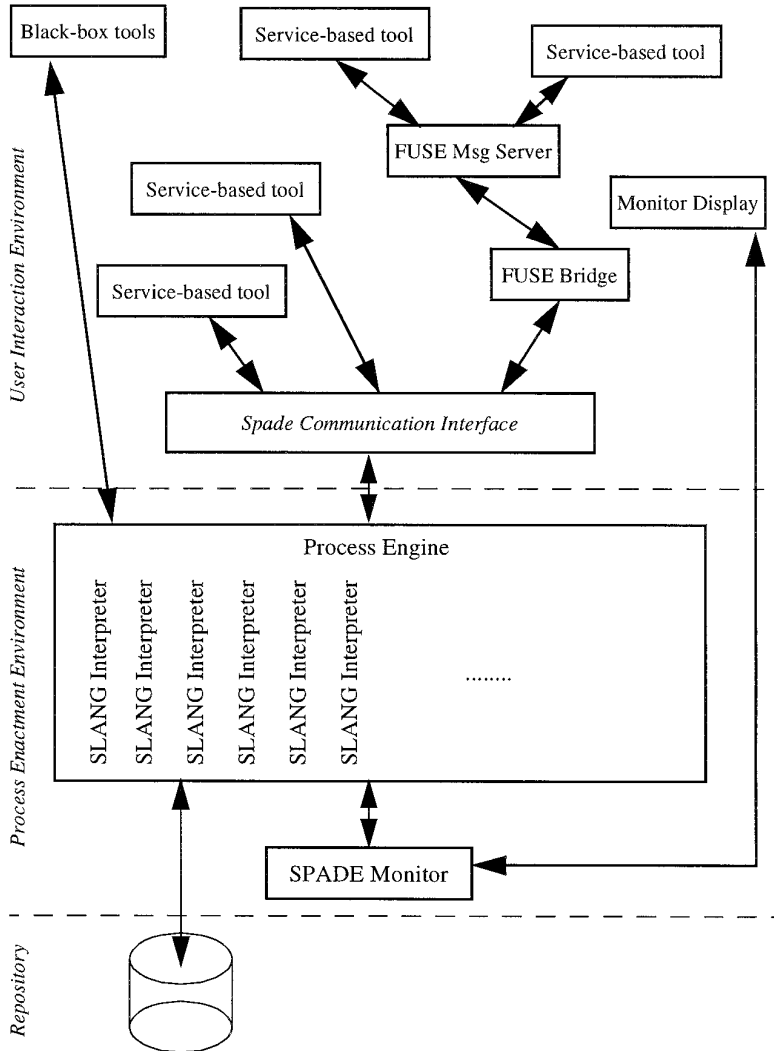


Fig. 3. SPADE-1 architecture.

Other Systems. Most PSEEs are based on architectures that are quite similar to those presented above: there is a server that is in charge of managing the environment database, while user interaction is supported by specialized clients and traditional software development tools that are integrated and controlled by the server and/or the clients. Significant variations are Process Weaver and Provenance. As discussed in Section 3.3, in Process Weaver each process model fragment is compiled and linked with the language interpreter. For each process fragment, an autonomous Unix process is created, which is controlled by usual operating system commands and services. Therefore, in Process Weaver there is no notion of a centralized process engine(s) (or monitor) as in other systems. Basically, each

process model fragment is embedded in a Unix process that is executed in a completely independent and autonomous way. Communication among different processes is achieved via shared files and by the possibility of starting the execution of a new process fragment (i.e., a new Unix process).

In Provence, a mirrored architecture is exploited [Krishnamurthy and Barghouti 1993]. The PSEE is not central in this architecture. Users operate their traditional tools based on the file system. A monitor detects relevant events at the operating system level (e.g., any file system operation) and communicates them to the process engine of the PSEE. Thus, the operations accomplished by users are transparently mirrored in the PSEE, and hence, Barghouti [1994] claims that it is possible to “separate process model enactment from process execution,” in order to enhance the degree of nonintrusiveness of the PSEE.

Evaluation. The architectures of the PIL run-time support of OIKOS, EPOS, and SPADE-1 partially share several common characteristics:

- (1) There are two distinct components: the process enactment environment and the user interface facility. The former offers the basic mechanisms to execute a process model, while the latter provides the interface with users.
- (2) In OIKOS and SPADE, the behavior of the user interface facility is not coupled with the paradigm of the PIL; that is, the behavior of the user interface does not depend on the semantics of the language. It is obtained through specific tools and by “programming” in PIL how these tools have to be managed.
- (3) The interaction with external tools is accomplished through tool integration facilities (Expo for OIKOS, BMS for EPOS, and FUSE for SPADE-1).
- (4) They all exploit a distributed approach, in which (at least some of) the components of the PSEE are distributed over a network of workstations.
- (5) There is only one centralized database in the environment.⁹

These are not the only alternatives that have been exploited in existing PSEEs. We still lack reference architectures, which can be used to assess and compare existing systems.

4.2 Interaction with Production Tools and Tool Integration Facilities

OIKOS. In Expo 2.0, there are two classes of tools: load-and-go and stand-by tools. Load-and-go tools (e.g., the cc compiler) are integrated by means of an envelope mechanism, and interact with the process model at the beginning and at the end of their execution. Stand-by tools interact with the process, exchanging messages and service requests.

⁹For a taxonomy of the different alternatives that can be exploited to build the repository of a PSEE, the reader can refer to Pueschel and Wolf [1993].

EPOS. Production tools are packaged (“enveloped”) as special task subtypes, with the actual Unix tool as an “external procedure.” The script implementing this procedure can include the invocation of messages for a tool integration facility such as BMS. A model instance (a leaf task instance) of such a type represents an actual tool invocation. An extra input “parameter” to this task provides return information on actual tool performance. Namely, this task can be reactivated many times to handle feedback from the tool. That is, such a task represents a small state machine.

SPADE-1. Interaction with external tools and tool integration facilities is achieved by two basic constructs in SLANG: the black transition and the user place. A *black transition* is used to start any external Unix program asynchronously. In this way, traditional Unix tools such as *vi* or *cc* can be directly invoked and controlled by the SLANG interpreter. They are seen as functions that provide results at the end of their execution. The filter discussed in Section 4.1 is used to control more sophisticated tools that offer a set of services through a programmatic interface (e.g., DEC FUSE). In this case, black transitions are used to send messages to the filter and from there to communicate with the tool integration facility. *User places* are used to store tokens that describe events that occurred in the user interaction environment and that were detected by the filter.

The integration of environments such as FUSE into SPADE-1 has caused substantial improvements to the tool integration facilities themselves. In particular, FUSE is basically a single user environment; that is, each user has its own instances of FUSE, and different instances (even of the same user) cannot communicate to each other. In SPADE, the process engines and the filter make it possible to reroute messages among different instances of FUSE. Also, it is possible to exchange messages between instances of heterogeneous tool integration facilities. For instance, it is possible to catch, convert, and manipulate a FUSE message, and then to forward it to a Tooltalk or an OLE2 tool.

Other Systems. Most environments adopt an integration strategy that exploits a tool integration facility. This is the case, for instance, of MERLIN (Sun Tooltalk), Process Weaver (HP BMS), and Articulator/Synervision (HP BMS).

The other emerging technology that is being used in PSEEs is OMG CORBA. For example, an interesting experimentation in this direction is being carried out in the E³ project [Baldi and Dall’Anese 1995].

Evaluation. In EPOS and SPADE-1, control integration is achieved through tool integration facilities, such as FUSE, BMS, and Tooltalk. The first problem that arises by using these products stems from the policies used to reroute messages among tools. Actually, these tool integration facilities assume that all tools are equivalent. Conversely, to integrate tools effectively with a PSEE, it must be possible to declare the PSEE as a special tool (an external “monitor”), capable of potentially receiving all

messages and dynamically determining which tools they shall be forwarded to.

A second problem is related to the integration of existing tools (a posteriori integration). The adoption of a tool integration facility is particularly well suited for achieving a priori control integration. Namely, tools have to be designed and built in order to make it possible for the PSEE to control and invoke their services through the tool programmatic interface. Notice that this strategy does not directly address the issue of providing data integration; that is, it does not ensure that data are consistently accessed by the PSEE and the tools. However, since these kinds of tools can notify the environment about the operations they are going to accomplish (e.g., opening a file), it is reasonably easy to check at the process engine level that accesses to data are consistently accomplished and pursued.

However, most existing tools do not offer a programmatic interface. It is possible only to launch them and then trigger their termination. The operations accomplished by the tool during its execution are not reported to the environment (e.g., an open-file operation issued by a traditional text editor). To integrate these kinds of tools, it is essential to offer mechanisms and features to detect accesses to the file system. For this reason, approaches such as the one used in Provence are also necessary.

In summary, tool integration facilities are effective means for providing integration, because they make it possible to monitor and control tool operations (including data accesses). They pose specific requirements on the structure and features of tools. On the other hand, low-level operating system mechanisms are needed to integrate those tools that do not offer any explicit “hook” in order to control and monitor their behavior.

4.3 Behavior of the PSEE

OIKOS. Historically, the first version of the OIKOS user interaction environment was built to monitor the contents of the tuple spaces. It was possible to view the contents of a tuple space, to add and remove tuples from the tuple space, and to order them in ascending, descending, or historical order. Recently, the user interaction environment has been updated to a full graphical style where the tuple space is represented by icons, and commands are directly applicable to the icons. The new user interface is driven by a client in charge of maintaining the consistency between the interface and the state of the tuple space and of reacting to users' requests.

EPOS. Before work can start, a production workspace of files must be established according to the given product/version/workspace description of a corresponding subproject. The process engine maintains a graphical display of the enacting task network and an agenda to guide the user. Tools can be invoked either directly by the process engine or manually by the user through the agenda.

SPADE-1. A software developer using SPADE-1 interacts with the environment through a set of tools. These tools are either existing produc-

tion tools that have been integrated in SPADE-1 or specific process tools that have been developed to guide and support the user. This latter category includes the process agenda and a simple shell interpreter that is able to accept a command from the user and to forward a corresponding message to the filter. Therefore, SPADE-1 has a nonintrusive approach that tries to keep the presence of the PSEE environment hidden. Also, tools like the agenda are completely decoupled from the PML; that is, their semantics and behavior are explicitly “programmed” in the process model being executed. These features have been introduced to enable the development of different user interaction environments without changing the SLANG semantics, the SLANG interpreter, and the SPADE-1 architecture [Bandinelli et al. 1994b].

Other Systems. In most existing environments, the user interface is derived directly from the semantics of the PML, or at least, there are specific elements of the user interface that are directly linked to the semantics of the PSEE. For instance, in Leu (the industrial follow-up of MELMAC) the user interface is made up of an agenda where each task represents a transition of the underlying Petri-net-based process model. Two systems that adopt an approach similar to SPADE-1 (i.e., the user interface is decoupled from the PML semantics) are Process Weaver and APPL/A.

Evaluation. The view of the process offered to process participants (software developers, project managers, etc.) is clearly a critical issue. In previous sections, we have pointed out some of the problems that need to be addressed. They can roughly be summarized with the following issues:

- (1) What are the correct metaphors that have to be used by the PSEE to interact with the user?
- (2) What is the correct level of abstraction to be used in the PSEE interface?
- (3) How is the PSEE interface built and controlled? Is it directly coupled with the PML (PIL) paradigm, or is it decoupled from it? Namely, are the metaphors and the abstraction level of the PSEE interface directly derived from the PIL semantics?

The issue of coupled versus decoupled user interfaces in PSEEs has been discussed in more detail by Bandinelli et al. [1994b]. The key issue is that in a coupled approach the user interface behavior is derived from the PIL semantics, while in a decoupled approach, it is “programmed” using the PIL. In the former case, the state of the user interface can be derived automatically from the state of the process enactment environment, but it is constrained by the semantics of the PIL. In the latter case, the state of the user interface has to be managed explicitly in the PIL specification, but the behavior of the user interface is not constrained by the semantics of the PIL.

EPOS is based on the coupled approach, whereas OIKOS and SPADE are based on the decoupled approach. It is not clear yet from our experience which is the correct approach to follow (coupled vs. decoupled) or if some intermediate solution is feasible and convenient. Most existing systems seem to prefer the coupled approach. Certainly, this is an open issue to be further addressed.

4.4 Data Integration

OIKOS. The OIKOS database is centralized, and every request is serialized. The service layer abstracts this view, allowing the creation of a set of independent repositories. Each repository is controlled by a service that provides operations to move documents from the repository to the workspace and vice versa, to lock documents, and to inspect the state of the repository. The physical database is split into two parts: the description of the documents is kept by Salad, while documents are stored as Unix files in the distributed file system.

EPOS. The client-server EPOS-DB stores the process model and maintains a versioned, nested, and cooperating transaction context. Products are stored as long fields (software documents are stored as files) under full control of the database. The database does not know anything about the inner contents of the files, so no “semantic” data integration can be offered. If products reside in internal development databases maintained by some CASE tools, EPOS cannot easily provide integrated product and process support.

Given that EPOS-PM is separated from EPOS-DB through a Prolog database interface, it should be possible to move to another (object-oriented) database platform. The versioning and transaction facilities may then have to be revised, although the version and transaction model are orthogonal to each other and to the underlying data model. Note, however, that versioning in EPOS-DB is provided at a basic (fragment) level inside the database, and not by the user on top of the database (as in the Damokles DBMS) or by the PCMS and ClearCase CM tools on top of some DBMSs.

SPADE-1. SPADE-1 stores all process models and process artifacts in an object-oriented repository implemented using the O_2 OODBMS. When information has to be made available to a tool that is not based on O_2 (e.g., any traditional Unix tool), it is checked out from the database and stored in the file system (a temporary workspace). The reverse operation occurs when the tool has completed its operation. Thus, data integration is achieved by sharing O_2 objects if the tool is built on top of the OODBMS. For traditional tools, it is necessary to explicitly check in and check out objects between the database and the file system. This is accomplished in the prologue and epilogue of the code associated with a SLANG black transition. The adoption of a sophisticated OODB to store all process artifacts (including SLANG process models) introduces some performance

problems, particularly when interpreting queries and when parts of a process model are recompiled after some change has been accomplished.

Other Systems. OIKOS, EPOS, and SPADE-1 are all based on the same approach: process artifacts are stored in a centralized object-oriented database. To exchange information with tools that are not integrated with the database, data are checked out and made available in temporary workspaces.

There are basically five other alternatives or variations to this approach, which are represented by the following systems:

- In Process Weaver, there is no database. The PSEE relies directly on the file system where all of the artifacts are stored.¹⁰
- In ALF, artifacts are stored in PCTE, which provides a transparent replacement of the Unix file systems [Canals et al. 1994].
- In Provence, a dual approach is adopted, since artifacts are stored in the file systems and are described by additional information stored in the PSEE repository (i.e., an object base).
- In Leu, a traditional relational database is used (Oracle) to complement and enrich file system features [Dinkhoff et al. 1994].
- In APPL/A, persistency is supported directly by the language [Sutton et al. 1995].

Evaluation. According to our experience, the problem of achieving effective data integration between production tools and the PSEE is one of the most critical issues. Most production tools rely on workspaces that are implemented using the file systems or proprietary database repositories. Thus, if the PSEE has its own database, expensive check-in/check-out operations must be defined to create and consistently manage temporary workspaces. Alternatively, it is necessary to detect the invocation of file system services at the operating system level.

We envisage two dimensions of the problem:

- (1) Strategically, we are interested in a standardization of technologies for PSEE databases, so that we can start migration of tools and diffusion of PSEE technology onto it. But what will this standard look like? Will it be some object-oriented language/interface (ODMG)? On the other hand, if we do not believe in any database standardization, one solution is *not* to have a PSEE database, as, for instance, in Process Weaver.
- (2) Pragmatically, we must develop some reasonable solutions to await a possible database standardization. For us, this has been the adoption of a specific database technology, such as the commercial O₂ in SPADE-1, the Salad prototype in OIKOS, and the proprietary EPOS-DB. This means that the PSEE is built around a dual approach, in which the central repository is temporarily and partially mirrored in file-based

¹⁰A research project is being carried out to integrate Process Weaver with the Adele system. The resulting environment is called WEADELE.

workspaces to support tool execution. This is similar to the approach adopted in Provence [Barghouti 1994].

We should also further explore the solution that is adopted in some systems (e.g., ALF), where the file system is transparently seen as part of the PSEE repository. In this way, existing tools do not need to be modified, while their accesses to the file system can be trapped and controlled by the PSEE.

4.5 Inconsistency Management and Workspace Synchronization

OIKOS. Workspaces are under control of a service. The files in a workspace are accessed by the tools integrated in Expo 2.0. According to this approach, consistency between the workspace state and its representation must be guaranteed by the service that manages the workspace. Traditional tools like *emacs* cannot be controlled fully by the service since they can create or manipulate files that are invisible to the tool envelope.

EPOS. A workspace manager is responsible for check-in/check-out of versioned files. A textual language (EPIT) is defined to enter new products easily, and to express the difference between an old and a new product structure. For process support to be effective inside of a transaction, the external workspace and the internal process model database must be “reasonably” in sync. The tools (especially a textual editor like *emacs*) can be instrumented to send BMS messages to the process model upon structural changes in the product. The planner can reconstruct task networks accordingly. To simulate a configuration management system with little process support between check-out and check-in, there are configuration management facilities for incremental check-in and for group check-in of changes (big “delta”) after a workspace transaction has been accomplished.

SPADE-1. SPADE-1 assumes that no operation on any process artifacts can be accomplished outside the control of the PSEE. SLANG and SPADE-1 architecture have been conceived to allow strict control over tool behavior. Clearly, this can be achieved only with those tools that provide specific “hooks” to notify and control their behavior, for example, service-based tools in the FIELD approach. More traditional tools such as *vi* are difficult to control and monitor, since they do not allow the PSEE to trigger and possibly prohibit the actions they are striving to accomplish.¹¹

Other Systems. An alternative to the approaches presented above is based on the provision of mechanisms at the operating system level to transparently see the file system as part of the repository and to define a workspace model explicitly.

Evaluation. In general, multiple descriptions of the same process entities may be stored in different parts of the PSEE or of its integrated tools.

¹¹Environments such as FUSE include versions of popular tools (e.g., *emacs*) that have been modified to be compliant with the requirement discussed so far. For instance, the editor notifies the environment that it is going to open a file.

For instance, a broadcast message server has an internal configuration table with tool and message definitions. This information must be kept consistent with the tool definitions and scripts used in the process model to invoke tools and to interact with them. These issues are not attacked by any of the three PSEEs we have developed or by any other approach we have studied.

As for workspace synchronization, this issue is obviously related to the problem of data integration. We can envisage intermediate solutions, such as instrumenting Unix-based tools and services to inform the PSEE about relevant file changes (partly done in EPOS and SPADE; see also Barghouti [1994]).

4.6 Distribution and Heterogeneity

OIKOS. Expo 2.0 is fully distributed on a network of Sun workstations. Process engines (i.e., instances of the Patè interpreter) and tools can be executed on any machine running under Expo 2.0.

EPOS. EPOS-DB is a client-server database. Advanced CASE tools exploit workspaces that are not easily integrated in the current architecture, because of the impossibility of monitoring or trapping data accesses.

SPACE-1. SPADE-1 exploits the client-server architecture of O_2 to distribute different processes over a network of workstations. Process engines, the filter, tools, and the O_2 server can be distributed over the network. Moreover, the adoption of a standard technology for interprocess communication (e.g., Unix sockets), and the availability of SPADE components on a variety of platforms make it possible to use a network of heterogeneous workstations as the operating environment. For instance, it is possible to have the filter and some FUSE instances running on a DECStation, and other SPADE components and FUSE instances running on Sun workstations.

Other Systems. Most existing PSEEs are based on a client-server approach. The issue of providing distributed and federated process support has been specifically addressed in the Oz project, which has defined and implemented policies and mechanisms to support integration of distributed and heterogeneous PSEEs by exploiting the treaty and summit metaphors [Ben-Shaul and Kaiser 1994].

Evaluation. In our three systems, elementary distribution is taken care of by standard local network facilities (e.g., RPC and sockets) and the services using them (NFS, client-server databases, the X-Window user interface, message servers). This allows client PSEEs to be run on heterogeneous machines, given that they can support the PIL in question. This is easier for SPADE-1 than for EPOS and OIKOS, since the former is based on more stable and widely available enabling technologies.

In general, we still lack a clear architectural and functional characterization of what a distributed and federated environment for process support

should look like. The only existing approach in this area is the Oz system, which provides a specific approach to distributing and federating PSEEs. Much work is still needed to understand the different policies that can be used to distribute and federate PSEEs. For instance, we might have a peer-to-peer cooperation with autonomous repositories, or a more abstract approach, where data and process features are transparently distributed over the network. We need to understand the basic mechanisms that the PSEE should offer to support the definition of these different process-specific policies.

4.7 Support for Measurement

None of the systems we are aware of provide specific support for measurement, apart from the ability to write specific PML fragments that explicitly collect data. Likewise, little support is provided to the process of establishing a measurement program [Fuggetta et al. 1996].

Evaluation. This is definitely one of the weakest areas in the entire domain of software process technology, in the sense that so far there is little effort to extend PSEEs to support the establishment and the operation of a measurement program. There are clearly a variety of issues involved:

- (1) Metrics cannot be defined once for all organizations. They have to be selected according to the goals, characteristics, and problems of each specific organization. We need methods and processes to create and operate a measurement program. Exploiting GQM is certainly an important step in this direction [Fuggetta et al. 1996].
- (2) The PSEE should provide the infrastructure to effectively support the measurement program. There are two extreme approaches we can envisage:
 - The PSEE/PML does not provide any specific means. The measurement program is defined by the process engineer according to the process models (at different levels of abstraction). The collection of data is accomplished by exploiting the tools on hand or by “programming” in PIL the operation needed to collect specific information. This is the solution offered by the authors’ systems. It does not require any specific extension of the basic PIL/PSEE technology.
 - The PSEE/PML includes specific concepts and aids to support the measurement program. For instance, the PSL/PIL constructs may be included to decorate (at the requirement and design levels) process steps with information on the metrics to be collected. In turn, the PSEE/PIL may offer specific built-in mechanisms to support and, whenever possible, to automate the collection of data.

This is another area where we feel careful experimentation is necessary to identify the most effective strategy.

4.8 Efficiency of the Implementation

OIKOS. The efficiency of Expo 2.0 is very high, because the network communication layer and all of the integration mechanisms are implemented in C and use basic (and efficient) Unix services. The efficiency of the database is low, because Salad requires a large amount of memory and is based on a logical deductive technology. The Patè interpreter is another bottleneck. Each instance of the Patè interpreter requires a large amount of swap area, independently of the number and the contents of the tuple space. The evaluation of the rules is slow, because no optimization technique has been used.

EPOS. EPOS-DB is a bit slow (e.g., 100 new objects generated per second). Presently, only type descriptors are cached at the clients, although general caching of process model items (as Prolog facts) has been demonstrated.

EPOS-PM is slow due to the underlying Prolog. However, we have optimized the process engine using triggers upon product change to support incremental rule evaluation and execution.

SPADE-1. The first prototype of SPADE-1 was developed to assess the problems related to the development of a PSEE. The major performance problems identified at that stage were the execution of dynamic queries, the database performance, and the Petri nets interpretation algorithm. The proliferation of Unix processes (e.g., the filter, process engines, the O₂ server) and the heavy usage of interprocess communication mechanisms do not have a strong impact on performance.

In SPADE-1, Versions 2.0 and 3.0, a major design and implementation effort has been devoted to amending the above-mentioned drawbacks. In particular, a mixed interpretation/compilation approach has been adopted, to improve the run-time performance while still preserving the reflexive features of the language. The interpretation algorithm has been rewritten to make it more incremental and to reduce the overhead related to transition guards evaluation.

Other Systems. This is a very difficult topic to discuss, since in most cases it is not considered at all in the literature.

Evaluation. Most authors claim that process activities are carried out on a timescale of days or even months. So the operations accomplished by the PSEE do not need to be particularly fast. This is just one side of the coin. Indeed, if the PSEE is used to control and integrate tools, or to support the interaction with the user, then performance becomes a critical issue. Consider this example: In SPADE-1 any message exchanged by tools may be trapped and reified in the Petri net. In this way, it is possible to react to any message according to process-specific policies. This is certainly a powerful facility since it makes it possible to flexibly coordinate and control tools and, through them, the interaction with their users [Di Nitto and Fuggetta 1995]. Most of these messages, however, are sent as a

consequence of a user's request. The user is therefore waiting for some reaction of the tool that, in turn, is waiting for some answer to the message it sent out. So there is a turnaround time that is composed of the time needed to deliver a message to SPADE-1 (e.g., the user via the editor wants to open a file), the interpretation of the Petri net fragment dealing with the message (e.g., does he or she play the role allowed to change this file?), and the delivery of the answer to the tool (e.g., you can open the file). In this case, the timescale is in the order of seconds (e.g., the delay a user can tolerate when he or she invokes an "open file" operation). The SPADE-1 infrastructure must consistently ensure that the turnaround time is in the same timescale. This affects the message exchange service, the process engine interpretation algorithm, and the database access facility.

This example is fairly representative of a wide set of cases and situations where performances do have a significant impact on the usability and effectiveness of the environment. Therefore, the claim that performance is not a critical issue can be only partially supported, depending on the role that the PSEE is requested to play in supporting users' activities.

4.9 A General Issue: Usage of Standard Enabling Technology

A PSEE is a complex computerized system that must integrate and provide a large variety of features and services, *particularly when used to support process enactment*. To build such a system (i.e., the PIL run-time support), it is worthwhile to reuse and exploit state-of-the-art *enabling technology* (e.g., OODBMSs and distributed operating systems) [Fuggetta 1993]. The usage of existing technology, however, poses several technical problems (discussed in detail in the next paragraphs) and one strategic issue.

In building SPADE-1, one of the strategic choices was not to reimplement functionalities that can be reasonably (even if not completely) offered by existing technology. Moreover, only commercially available products and technology were considered (in particular, the O₂ OODBMS, FUSE, and C++ as the implementation language). In SPADE-1, therefore, there was a deliberate choice to opt for a more mature, efficient, and interoperable technology, with the price being less advanced features and functionalities to base the development of the PSEE (the only exception is, partially, O₂). On the contrary, in OIKOS and EPOS the choice, at least in some cases, was to opt for a more advanced technology (e.g., EPOS-DB and Prolog, and the Salad deductive database), even if this could cause problems with the exploitability at the industrial level.

These two strategies represent a basic trade-off that we (as a research community) are facing:

- (1) The development of PSEEs can greatly benefit from the more recent research in domains such as databases or programming environments. This would facilitate the implementation of the advanced features required by PSEEs and PMLs.
- (2) If we want to use PSEEs in real production environments, we need to consider the issues of efficiency, interoperability, and compatibility.

This would demand more consolidated technologies, which could be less effective in prototyping, building, and testing advanced PSEE/PML features.

We believe that the need for experimenting with PSEEs in real settings is so relevant that it makes the second option preferable.

5. EXPERIENCES

5.1 Reference Examples

OIKOS. The standard ISPW-6 example has been completely specified in Limbo and implemented in Patè. Work is in progress to complete the specification of the ISPW-7 and ISPW-9 examples.

EPOS. The ISPW-6 example has been completed. The development of the ISPW-7 and ISPW-9 examples is in progress.

SPADE-1. SLANG has been used to model reference examples such as ISPW-6, ISPW-7, and ISPW-9.

5.2 Internal Experiments

OIKOS. A large set of small examples has been used to test Limbo, Patè, and Expo 2.0. Among them, the most relevant are

- a simple development environment based on the Cleanroom approach; and
- part of the KAOS requirement elicitation model, in cooperation with the University of Louvain, Louvain, Belgium.

EPOS. Several internal examples have been developed, with work going on to cover parts of EPOS itself (versioned product archive, change request cycle, cooperation). Use of EPOS to support real industrial projects is not foreseen other than as a demonstrator.

SPADE-1. SLANG and SPADE-1 have been used to build several demonstration processes, each of them aimed at evaluating different features of the system. An integrated and comprehensive case study has been recently completed [Bandinelli et al. 1995; 1996b].

5.3 Industrial Projects

OIKOS. A large part of the PSS-05 model of the European Space Agency has been specified and partly implemented in cooperation with Intecs Sistemi, a software house operating in Pisa, Italy. Part of a proprietary development methodology based on IDEF-0 has been modeled in cooperation with Tecsiel, another software house operating in Pisa.

EPOS. Several external cases have been simulated: parts of the Cleanroom approach and some industrial cases that are under way (from

PROTEUS and REBOOT ESPRIT projects, Sysdeco Consulting, and Siemens Defense systems).

SPADE-1. SLANG has been used to model industrial processes in a large telecomm company in Italy [Bandinelli et al. 1995]. SPADE-1, Version 3.0, has been released to a few companies within the context of two ESPRIT projects. They are using SPADE-1 to support different processes, including a distributed configuration management system for C++ libraries and a complete software process based on Boehm's Spiral Model.

5.4 Other Systems

There have been several experiences encountered in the research and industrial community. Clearly, it is impossible to discuss them in detail here. The reader may refer to Fuggetta and Picco [1994], which offers an annotated bibliography of several interesting works.

An additional source of information is the set of solutions to the ISPW process example. One of the major goals achieved by the workshop is the definition of a standard example, called the *core problem*, which is referred to during the workshop as a common reference point. There are several versions of this example, which have been used by many researchers to assess and compare the features and characteristics of PSEEs and PMLs [Kellner et al. 1990].

5.5 Evaluation

We must admit that our experiences in using research prototypes in real industrial settings are still limited. Reference examples have proved to be useful for accomplishing a preliminary assessment of each PSEE, for comparing different solutions, and for identifying drawbacks and advantages. Nonetheless, they are insufficient to provide an effective evaluation of the technology in real comprehensive cases. We can summarize here the problems we encountered in applying OIKOS, EPOS, and SPADE-1 in real industrial settings:

- (1) Business organizations are quite immature as far as processes are concerned. In many cases, they do not perceive the importance of making processes explicit. In other cases, they do not have the resources to invest in process improvement.
- (2) Most of the technologies we have been developing are mainly oriented toward providing enactment assistance, while in many cases the most critical requirement is support for process comprehension, analysis, and documentation.
- (3) We still do not know how to introduce a new technology effectively and incrementally in a production environment. A technology such as PSEEs may have a very strong impact or demand for a complex "transient" process. We need to reduce and limit the cost of this transition and the corresponding perturbation on ongoing activities.

- (4) We need to better understand how to evaluate the impact that software process analysis and automation can have on real production environments.

According to other published case studies and anecdotes, it appears that these problems are related to the rather low maturity level of most organizations. A possible correlation between the maturity of an organization and the effectiveness of process technology can be summarized as follows:

- Organizations at Level 1 or 2 of the CMM [Paulk et al. 1995] are more interested in PSLs and tools that help them to assess and describe how their organization works. They are also interested in evaluating and exploring different alternatives before processes are used in actual software development activities. So simulation and analysis facilities are certainly needed.

Since most existing PMLs are just PILs, their adoption is quite difficult. These organizations are simply not able to proceed to the implementation of process support and, therefore, cannot exploit the features of this technology.

- In more mature organizations, existing technology can be much more effective. Organizations at Level 3 or higher have detailed and described processes (even if often just informally). PILs can be used to better detail these processes, to improve their applicability, and to increase the ability to track process performances [Bandinelli et al. 1995]. Moreover, since processes are already in use, PSEEs can be exploited to implement process support.

Certainly, much experimental work is still needed to consolidate and further assess these observations.

6. RELATED WORK ON EVALUATING PSEEs

Papers that describe experiences in building and using software engineering environments are now easy to find. The field started more than 10 years ago. Many research prototypes have now become commercial systems, or at least, the concepts underlying their design are well understood and have been exploited. However, the state of the art in PSEEs, where the main emphasis is on the role of process modeling, is still immature. As we discussed in the introduction, we have just reached the point where industrial application of existing products has been started. Several products are available, and some research initiatives have reached the critical stage in which prototypes begin to provide feedback to the designers. What we really lack is a body of experiences and lessons learned to consolidate our initial experimentation.

We have carefully searched the current literature with limited success. Many papers describe a single system, often based on the “show and tell” approach. Comparisons can be found with specific aspects of the system,

such as the PML used, hints on the architecture, and so on. Other efforts aim to define a coherent and, possibly, standard terminology. This is typical of a young scientific field that experiences a rapid growth. A general overview of PMLs and PSEEs is presented by Huff [1996] and Garg and Jazajeri [1996], respectively.

An assessment exercise similar to ours is described by Lonchamp [1994]. Lonchamp submitted a questionnaire to different research groups involved in the development of PSEEs. The questionnaire identifies several key issues and questions. The answers have been collected and summarized in several tables, by associating to each system one or more keywords identifying the answer to a given question. Certainly, it is very difficult to synthesize via keywords the properties and characteristics of the PSEEs being assessed. Moreover, different groups might interpret the same question in different ways. Finally, the tables partially overlap, and therefore, it may be difficult to get a general overview of the results.

An interesting evaluation paper appears in the *Proceedings of the 8th International Software Process Workshop* [Barghouti and Feiler 1993]. In this paper, seven systems demonstrated at the workshop are compared through the use of a grid that gives an overall view of the efforts. In general, these workshops are characterized by the effort to mix experiences coming from Europe, the United States, and Japan. The approach that has been taken to conduct the comparison is based on a set of questions that are related mainly to the use of the systems, rather than to their architecture and underlying principles. In particular, the authors identify the following main questions:

- (1) What is the usability of the system?
- (2) Who is the user of the system, and how does he or she interact with it?
- (3) What were the main technical problems encountered while building the system?
- (4) How easy is it for people to learn the system?
- (5) Is the system really used?
- (6) What is missing?
- (7) How easy is the system to use?
- (8) Who is in control, the user or the system?
- (9) How are errors handled?
- (10) How are models created and evaluated?
- (11) What is the support for change?
- (12) What is the support for the configuration management of process models?
- (13) What is the level of abstraction of the concepts used in the process modeling language?

These questions raised a very interesting discussion, but they do not allow the systematic evaluation of existing systems and, more importantly, they

do not address the details of several technical issues related to the development of PSEEs/PMLs (e.g., data and control integration).

Another source of comments and experiences is the outcome of the Promoter working group on Software Processes. This working group has been established by the European Union in the Third Framework Programme. It aims at facilitating and promoting the exchange of experiences and results in the software process field. The final goal is the establishment of a European research and industrial community in this area. There are basically two initiatives that have been carried out by this group:

- (1) a series of workshops on Software Process Technology that was started in 1991 and that has made possible the creation of a forum for discussing and comparing the different research efforts carried out in Europe (see Fuggetta et al. [1991], Derniame [1992], Warboys [1994], and Schäfer [1995]); the fifth edition of the workshop was held in Autumn 1996 in France [Montangero 1996]; and
- (2) a book that summarizes the major projects carried out in Europe [Finkelstein et al. 1994].

Experiences in modeling a large number of processes can be found in Scacchi and Mi [1993], which reports on the outcome of a modeling activity that has covered more than 30 production processes, including both “as-is” processes and those that represent new or alternative “to-be” processes. An interesting comment on the distance between the real-world process and the computerized process model can be found in Frailey [1993]. Here, the domain is slightly different, being more oriented toward concurrent engineering. In the “Lessons learned and conclusions” section of the paper, Frailey writes,

Processes are much more difficult to define than one first imagines. When forced to put things down on paper, the definition teams found that they did not always understand things the same way. The ensuing resolution process was very educational for all concerned, although at times it seemed like much ado about very little.

This observation puts new light on the importance of the higher phase of the process-modeling metaprocess.

Statements very close to those expressed in our work can be found in Sutton [1993]. In the introduction, Sutton writes,

In this paper I discuss properties of software process and process programs, analyze issues and conditions that affect the development and use of process programs, and indicate opportunities, limitations, and trade-offs that arise from these issues and conditions.

Among the properties that a process program must have, Sutton mentions the importance of fitness (i.e., the ability of people to follow a process definition) and precision. Finally, process modeling is characterized as an intrinsically evolutionary activity, where both static and dynamic changes must be properly taken into account.

7. EVALUATION OF THIS ASSESSMENT

Evaluating PSEEs is a very difficult task for a variety of reasons:

- (1) PSEEs and PMLs are characterized by many features and components. There are several technical aspects to take into account, ranging from the paradigm of the language to the mechanisms used to interface tools. All of these different aspects contribute to the quality and effectiveness of a specific approach, even if none of them is more relevant and critical than the others.
- (2) We lack a reference model and architecture that can be used to analyze and assess existing PSEEs. Authors present their approaches according to their own preferences and technical viewpoints, and it is consequently very difficult to accomplish a comparative analysis with the state of the art and between different systems.
- (3) The application of process technology, even in trial cases, is expensive and time consuming. Moreover, the organization should be reasonably mature in order to effectively exploit the technology.
- (4) The evaluation of the impact of process technology on real industrial processes has to be accomplished by taking into account different factors, ranging from economical benefits to social and human aspects.

For these reasons, it is quite difficult for a single organization or research group to carry out a comparative assessment covering different systems. The only reasonable way to pursue this activity is to create self-assessment procedures that make it possible for each single organization to produce an evaluation autonomously. To achieve this objective, we believe it is necessary to pursue the following initiatives:

- The grid needs further refinements for early process lifecycle phases, user interfaces, and metaprocesses. Moreover, it is essential to *relate and analyze PML and PSEE by jointly considering linguistics and architectural issues*. We realize that in some cases the sharp distinction between PML and PSEE that we introduced in the grid might seem a bit unnatural.
- The ISPW examples we use to compare existing systems should be made more precise and general, in order to fully cover different process situations and to properly solicit and benchmark PSEE functionalities and features.
- It is essential to create a reference architecture (or alternative architectures) to make it easier to identify and compare the high-level structure of PSEEs.
- The self-assessment procedure should be conceived in order to guarantee a minimum level of homogeneity and to reduce the bias of the evaluator.

This article is a first step in this direction. Two other important contributions are the two international workshops on PSEE architectures, which were held in Milano, Italy, in March 1995 [Bandinelli 1996] and in Irvine,

California, in February 1996. In these workshops, different PSEEs have been presented and compared in order to progress toward a common understanding and identification of a tentative reference architecture for PSEEs.

8. CONCLUSIONS

This article has presented an assessment grid for PSEEs. Three European PSEEs and other existing approaches have been characterized according to this grid. It has also been used to organize and illustrate lessons learned and open issues in the PSEE area. The grid has been proven satisfactory, but it needs extensions and improvements for early process lifecycle phases, user interfaces, and metaprocesses. The emphasis of this article is on providing an initial grid structure and on accomplishing an initial application of the grid to evaluate its suitability.

We strongly believe that it is important to look critically at our R&D activities and to assess them according to the collected experiences by early application of this technology to real (or realistic) software processes. The problems and issues that need to be addressed in the near future can be summarized by the following questions:

- Do we need one single PML or multiple, specialized PMLs?
- What is the right abstraction level for these different languages?
- Do we need specific paradigms (e.g., object orientation for product modeling), and which paradigm should be used for which kind of PML?
- What is the role of the PSEE database?
- What is the architecture of a PSEE, and what are its main components?
- What should the PSEE interface look like?
- How do we build the infrastructure (or run-time support) for PSEEs?
- What are the requirements that the development of PSEE poses to other technologies, such as distributed systems, databases, and tool integration facilities?
- How can and should a PSEE take advantage of the existing enabling technologies?
- How do we evaluate the impact that process technology can have on real production processes?
- How do we manage the introduction of this technology?
- ... *and many others!*

We have tried to answer some of these questions by retrospectively looking back at the work done in the last 3–5 years. We believe that it is necessary to exchange information and opinions on these problems in order to move the field toward a higher level of maturity and to offer a real breakthrough to software development organizations.

ACKNOWLEDGMENTS

Thanks go to colleagues in the PROMOTER project, and to the local teams behind OIKOS, EPOS, and SPADE. We also want to acknowledge the useful comments and contributions of the anonymous reviewers.

REFERENCES

- ABDEL-HAMID, T. K. AND MADNICK, S. 1991. *Software Project Dynamics: An Integrated approach*. Prentice-Hall, Englewood Cliffs, N.J.
- AMBRIOLA, V., CIANCARINI, P., AND MONTANGERO, C. 1990. Enacting software processes in OIKOS. In *Proceedings of the 4th ACM SIGSOFT Symposium on Software Development Environments (SDE 4)*. ACM, New York, 183–192.
- AMBRIOLA, V., CIGNONI, G. A., AND MONTANGERO, C. 1994. The OIKOS services for object management in the software process. In *Proceedings of the 3rd European Workshop on Software Process Technology (EWSPT 94)*. Lecture Notes in Computer Science, vol. 772. Springer-Verlag, New York, 2–14.
- ARBAOUI, S. AND OQUENDO, F. 1994. PEACE: Goal-oriented logic-based formalism for process modeling. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- BALDI, M. AND DALL'ANESE, T. 1995. A distributed object-based system for process model enactment. Tech. Rep., Dipartimento di Automatica e Informatica, Politecnico di Torino, Turin, Italy.
- BANDINELLI, S. 1996. Report on the workshop on Software Process Environment architecture. *Softw. Process-Improv. Pract.* 2, 1 (Mar.).
- BANDINELLI, S., CERI, S., AND FELDER, M. 1996a. TANGO: A notation for describing advanced transaction models. In *Proceedings of the International Conference on Information Systems Analysis and Synthesis* (July).
- BANDINELLI, S., DI NITTO, E., AND FUGGETTA, A. 1994a. Policies and mechanisms to support process evolution in PSEEs. In *Proceedings of the 3rd International Conference on the Software Process (ICSP 3)*. IEEE Computer Society, Washington, D.C.
- BANDINELLI, S., DI NITTO, E., AND FUGGETTA, A. 1996b. Supporting cooperation in software development. *IEEE Trans. Softw. Eng.* 22, 12 (Dec.), 9–20.
- BANDINELLI, S., DI NITTO, E., FUGGETTA, A., AND LAVAZZA, L. 1994b. Coupled vs. decoupled user interfaces in PSEEs. In *Proceedings of the 9th International Software Process Workshop (ISPW 9)*. IEEE Computer Society, Washington, D.C.
- BANDINELLI, S., FUGGETTA, A., GHEZZI, C., AND LAVAZZA, L. 1994c. SPADE: An environment for software process analysis, design, and enactment. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- BANDINELLI, S., FUGGETTA, A., LAVAZZA, L., LOI, M., AND PICCO, G. 1995. Modeling and improving an industrial software process. *IEEE Trans. Softw. Eng.* 21, 5 (May).
- BARGHOUTI, N. S. 1994. Separating process model enactment from process execution in Provence. In *Proceedings of the 9th International Software Process Workshop (ISPW-9)*. IEEE Computer Society, Washington, D.C.
- BARGHOUTI, N. S. AND FEILER, P. H. 1993. Demonstration experience report sessions summary. In *Proceedings of the 8th International Software Process Workshop (ISPW 8)* (Wadern, Germany). IEEE Computer Society Press, Los Alamitos, Calif., 2–5.
- BEN-SHAUL, I. Z. AND KAISER, G. 1994. A paradigm for decentralized process modeling and its realization in the Oz environment. In *Proceedings of the 16th International Conference on Software Engineering* (May). IEEE Computer Society, Washington, D.C.
- BRUYNNOGHE, F., GREENWOOD, R. M., ROBERTSON, I., SA, J., AND WARBOYS, B. C. 1994. PADM: Towards a total process modeling system. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- CANALS, J., BOUDJLIDA, N., DERNIAME, J. C., GODART, C., AND LONCHAMP, J. 1994. ALF: A framework for building process-centered software engineering environments. In *Software*

- Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- CHIMENTI, D., GAMBA, R., KRISHNAMURTHY, R., NAQVI, S., TSUR, S., AND ZANIOLO, C. 1990. The LDL system prototype. *IEEE Trans. Knowl. Data Eng.* 2, 1 (Mar.).
- CONRADI, R., FERNSTRÖM, C., AND FUGGETTA, A. 1994a. Concepts for evolving software processes. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- CONRADI, R., LIU, C., AND HAGASETH, M. 1995. Planning support for cooperating transactions in EPOS. *Inf. Syst.* 20, 4 (June), 317–326.
- CONRADI, R., HAGASETH, M., LARSEN, J.-O., NGUYEN, M. N., MUNCH, B. P., WESTBY, P. H., ZHU, W., JACCHERT, M. L., AND LIU, C. 1994b. EPOS: Object-oriented cooperative process modeling. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- CURTIS, B., KELLNER, M., AND OVER, J. 1992. Process modeling. *Commun. ACM* 35, 9 (Sept.).
- DERNAME, J. C., Ed. 1992. *Proceedings of EWSPT 92 (2nd European Workshop on Software Process Technology)*. Lecture Notes in Computer Science, vol. 635. Springer-Verlag, New York.
- DI NITTO, E. AND FUGGETTA, A. 1995. Integrating process technology and CSCW. In *Proceedings of the 4th European Workshop on Software Process Technology*. Lecture Notes in Computer Science, vol. 913. Springer-Verlag, New York.
- DINKHOFF, G., GRUHN, V., SAALMANN, A., AND ZIELONKA, M. 1994. Business process modeling in the workflow management environment LEU. In *Proceedings of the 13th International Conference on the Entity-Relationship Approach*. Lecture Notes in Computer Science, vol. 881. Springer-Verlag, New York.
- FEILER, P. AND HUMPHREY, W. 1993. Software process development and enactment: Concept and definitions. In *Proceedings of the 2nd International Conference on the Software Process (ICSP 2)* (Berlin, Germany). IEEE Computer Society, Washington, D.C.
- FERNANDEZ, M. F. AND ZDONIK, S. B. 1989. Transaction groups: A model for controlling cooperative work. In *3rd International Workshop on Persistent Object Systems, Their Design, Implementation and Use* (Jan.). 341–350.
- FERNSTRÖM, C. 1993. Process Weaver: Adding process support to UNIX. In *Proceedings of the 2nd International Conference on the Software Process*.
- FINKELSTEIN, A., KRAMER, J., AND NUSEIBEH, B., Eds. 1994. *Software Process Modeling and Technology*. Research Studies Press, London, U.K.
- FRAILEY, D. J. 1993. Concurrent engineering and the software process. In *Proceedings of the 2nd International Conference on the Software Process (ICSP 2)* (Berlin, Germany). IEEE Computer Society Press, Los Alamitos, Calif.
- FUGGETTA, A. 1993. A classification of CASE technology. *IEEE Computer* 26, 12 (Dec.), 25–38.
- FUGGETTA, A. AND GHEZZI, C. 1994. State of the art and open issues in PSEEs. *J. Syst. Softw.* 26, 1 (July), 53–60.
- FUGGETTA, A. AND PICCO, G. P. 1994. An annotated bibliography on software process improvement. *ACM SIGSOFT Softw. Eng. Notes* 19, 4 (July), 66–68.
- FUGGETTA, A. AND WOLF, A., Eds. 1996. *Trends in Software: Software Process*. Vol. 4. John Wiley and Sons, New York, 160.
- FUGGETTA, A., CONRADI, R., AND AMBRIOLA, V., Eds. 1991. *Proceedings of the 1st European Workshop on Software Process Modeling*. AICA (Italian National Association for Computer Science), Milan, Italy.
- FUGGETTA, A., GHEZZI, C., MANDRIOLI, D., AND MORZENTI, A. 1993. Executable specification with dataflow diagrams. *Softw. Pract. Exper.* 23, 6 (June), 629–693.
- FUGGETTA, A., LAVAZZA, L., MORASCA, S., CINTI, S., OLDANO, G., AND ORAZI, E. 1996. Applying G/Q/M in an industrial software factory. Tech. Rep., CEFRIEL, Milano, Italy.
- GARLAN, D. AND SHAW, M. 1993. An introduction to software architecture. In *Advances in Software Engineering and Knowledge Engineering*, V. Ambriola and G. Tortora, Eds. World Scientific Publishing Co.

- GARG, P. AND JAZAJERI, M. 1996. PSEEs: A grand tour. In *Trends in Software: Software Process*, A. Fuggetta and W. Wolf, Eds. John Wiley and Sons, New York.
- GARG, P., MI, P., PHAM, T., SCACCHI, W., AND THUNQUEST, G. 1994. The SMART approach for software process engineering. In *Proceedings of the 16th International Conference on Software Engineering*. IEEE Computer Society, Washington, D.C.
- GRUHN, V. AND JEGELKA, R. 1992. An evaluation of FUNSOFT nets. In *Proceedings of the 2nd European Workshop on Software Process Technology (EWSPT 92)*. Lecture Notes in Computer Science. Springer-Verlag, New York.
- HUFF, K. 1996. Software process modeling. In *Trends in Software: Software Process*, A. Fuggetta and W. Wolf, Eds. John Wiley and Sons, New York.
- HUFF, K. AND LESSER, V. 1989. A plan-based intelligent assistant that supports the software development process. In *Proceedings of the 3rd Software Engineering Symposium on Practical Software Development Environments*.
- IEEE. 1994. Special issue on measurement-based process improvement. *IEEE Softw.* 11, 4 (July).
- IIDA, H., MIMURA, K., INOUE, K., AND TORII, K. 1994. Hakinowa: Monitor and navigation system for cooperative software development. In *Proceedings of the 3rd International Conference on the Software Process*.
- JUNKERMANN, G., PUESCHEL, B., SCHÄFER, W., AND WOLF, S. 1994. MERLIN: Supporting cooperation through a knowledge-based environment. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- KAISER, G., POPOVICH, S., AND BEN-SHAUL, I. Z. 1993. A bi-level language for software process modeling. In *Proceedings of 15th International Conference on Software Engineering (ICSE 15)* (Baltimore, Md.). IEEE Computer Society, Washington, D.C.
- KATAYAMA, T. 1989. A hierarchical and functional software process description and its enactment. In *Proceedings of the 11th International Conference on Software Engineering*.
- KELLNER, M., FEILER, P., FINKELSTEIN, A., KATAYAMA, T., OSTERWEIL, L., PENEDO, M., AND ROMBACH, H. D. 1990. Software process modeling example problem. In *Proceedings of the 6th International Software Process Workshop*. IEEE Computer Society Press, Los Alamitos, Calif., 19–29.
- KORTH, H., KIM, W., AND BANCILHON, F. 1985. A model of CAD transactions. In *Proceedings of the 11th International Conference on Very Large Databases*. VLDB Endowment, Saratoga, Calif., 25–33.
- KRAMER, J. AND MAGEE, J. 1990. The evolving philosophers problem: Dynamic change management. *IEEE Trans. Softw. Eng.* 16, 11 (Nov.), 1293–1306.
- KRISHNAMURTHY, B. AND BARGHOUTI, N. 1993. Provence: A process visualization and enactment environment. In *Proceedings of 4th European Software Engineering Conference (ESEC 93)*. Lecture Notes in Computer Science, vol. 717. Springer-Verlag, New York, 451–465.
- LONCHAMP, J. 1993. A structured conceptual and terminological framework for software process engineering. In *Proceedings of the 2nd International Conference on the Software Process (ICSP 2)* (Berlin, Germany). IEEE Computer Society Press, Los Alamitos, Calif.
- LONCHAMP, J. 1994. An assessment exercise. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- MADHAVJI, N. AND SCHÄFER, W. 1991. Prism-methodology and process-oriented environment. *IEEE Trans. Softw. Eng.* 17, 12 (Dec.), 1270–1283.
- MI, P. AND SCACCHI, W. 1990. A knowledge-based environment for modeling and simulating software engineering processes. *IEEE Trans. Knowl. Data Eng.* 2, 3 (Mar.).
- MONTANGERO, C. 1995. In favour of a coherent process coding language. In *Proceedings of the 4th European Workshop on Software Process Technology*. Lecture Notes in Computer Science, vol. 913. Springer-Verlag, New York.
- MONTANGERO, C., Ed. 1996. *Proceedings of the 5th European Workshop on Software Process Technology (EWSPT 96)* (Nancy, France, Oct.). Lecture Notes in Computer Science, vol. 1149. Springer-Verlag, New York.

- MONTANGERO, C. AND AMBRIOLA, V. 1994. OIKOS: Constructing process-centered SDEs. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- PAULK, M. C., WEBER, C. V., CURTIS, B., AND CHRISSIS, M. B., Eds. 1995. *The Capability Maturity Model: Guidelines for Improving the Software Process*. Software Engineering Inst., Carnegie-Mellon Univ., Pittsburgh, Pa., and Addison-Wesley, Reading, Mass.
- PUESCHEL, B. AND WOLF, S. 1993. Architectural support for distributed process-centered software development environments. In *Proceedings of the 8th International Software Process Workshop (ISPW 8)* (Dagstuhl, Germany, Mar.). IEEE Computer Society, Washington, D.C.
- RABBI, S. AND OQUENDO, F. 1994. PEACE: Goal-oriented logic-based formalism for process modeling. In *Software Process Modeling and Technology*, A. Finkelstein, J. Kramer, and B. Nuseibeh, Eds. Research Studies Press, London, U.K.
- SCACCHI, W. AND MI, P. 1993. Experiences in the modeling, analysis, and simulation of formalized software processes. In *Proceedings of the 8th International Software Process Workshop (ISPW 8)* (Dagstuhl, Germany). IEEE Computer Society Press, Los Alamitos, Calif.
- SCHÄFER, W., Ed. 1995. *Proceedings of the 4th European Workshop on Software Process Technology (EWSPT 95)* (Noordwijkerhout, The Netherlands, Apr.). Lecture Notes in Computer Science, vol. 913. Springer-Verlag, New York.
- SCHÄFER, W. AND WOLF, S. 1994. Cooperation patterns for process-centered software development environments. Tech. Rep., Dept. of Computer Science, Univ. of Dortmund, Dortmund, Germany.
- SKARRA, A. 1989. Concurrency control for cooperating transactions in an object-oriented database. *SIGPLAN Not.* 24, 4 (Feb.), 466–473.
- SNOWDON, R. 1992. An example of process change. In *Proceedings of the 2nd European Workshop on Software Process Technology (EWSPT 92)*. Lecture Notes in Computer Science, vol. 635. Springer-Verlag, New York.
- SUTTON, S. 1993. Opportunity, limitations, and trade-offs in process programming. In *Proceedings of the 2nd International Conference on the Software Process (ICSP 2)* (Berlin, Germany). IEEE Computer Society Press, Los Alamitos, Calif.
- SUTTON, S., HEIMBIGNER, D., AND OSTERWEIL, L. 1995. APPL/A: A language for software process programming. *ACM Trans. Softw. Eng. Methodol.* 4, 3 (July), 221–286.
- WARBOYS, B., Ed. 1994. *Proceedings of the 3rd European Workshop on Software Process Technology (EWSPT 94)*. Lecture Notes in Computer Science, vol. 772. Springer-Verlag, New York.

Received December 1994; revised May 1995 and August 1996; accepted February 1997